

SWARM-BASED MONOCULAR VISION SCANNING ROBOT

Zhuyu Dong, Weijie Chen, Junjie Ouyang, Qingzhou
Zhang, Chi Zou

4th Year Project Final Report

Department of Electronic &
Electrical Engineering

UCL

Supervisor: Dr Oliver Haderl

21 April 2026

I have read and understood UCL's and the Department's statements and guidelines concerning plagiarism.

I declare that all material described in this report is my own work except where explicitly and individually indicated in the text. This includes ideas described in the text, figures and computer programs.

I acknowledge the use of OpenAI GPT-5.4 and GPT-5.3 Codex (Publisher: OpenAI, URL: <https://chatgpt.com/>) to support code assistance during implementation, idea expansion and organisation during project planning, and translation, grammatical polishing, and length reduction during the writing of this report. The main technical ideas, system design decisions, implementation work, and core content of the project were developed and verified by the authors.

This report contains 37 pages (excluding this page and the appendices) and 11055 words.

Signed: Zhuyu Dong, Weijie Chen, Junjie
Ouyang, Qingzhou Zhang, Chi
Zou

Date: 21 April 2026

Swarm-Based Monocular Vision Scanning Robot

Zhuyu Dong, Weijie Chen, Junjie Ouyang, Qingzhou Zhang, Chi Zou

Contents

1	Introduction and Literature Review	4
1.1	Related Work on Low-Cost SLAM and Multi-Robot Exploration	4
2	Theoretical, Mathematical and Scientific Background	6
2.1	Relative Pose Estimation of the Robot	6
2.1.1	Coordinate Systems and the Chain Rule	6
2.1.2	Coordinate Transformation Update Based on Velocity and Attitude . .	6
2.2	Image-to-Point Cloud Reconstruction	7
2.2.1	Monocular Depth Estimation	7
2.2.2	Relative Depth and Metric Depth	7
2.2.3	Pinhole Camera Model	8
2.2.4	Back-Projection Geometry	8
2.2.5	Camera Calibration	8
2.3	Point Cloud Merging	9
2.3.1	Registration	9
2.3.2	Surface Fusion	10
2.4	2D Map Generation and Cost Modeling	11
2.4.1	Two-Dimensional Grid Map and Its Role	11
2.4.2	Ground Layer Selection	11
2.4.3	Eigenvalues and Eigenvectors for Geometric Characteristics	12
2.5	Path Planning	14
2.5.1	Map Representation and Frontier Definition	14
2.5.2	Robot Motion and Coordinate Model	14
2.5.3	Path Planning Method (A^*)	14
2.5.4	Collision Detection Model	14
2.5.5	Perception Models and Information Acquisition	14
2.5.6	Observation Overlap Constraint	15
2.5.7	Single-Robot Decision-Making Model	15
2.5.8	Multi-Robot Collision Avoidance	15
2.5.9	Field-of-View Constraints and Cooperative Avoidance	15
3	Methods, Setups and Software Implementation	15
3.1	Architecture Overview	15
3.2	Robot Platform	17
3.2.1	Second-Generation Platform	17
3.2.2	Bill of Materials and Cost Estimate	19
3.3	RTOS-Based Robot Control Architecture	19
3.4	Image-to-Point Cloud Reconstruction	20
3.4.1	Pipeline Overview	20

3.4.2	Image Input and Pre-processing	21
3.4.3	Depth Model Selection	21
3.4.4	Camera Calibration Procedure	21
3.4.5	Point Cloud Post-processing	21
3.4.6	Runtime Performance	22
3.4.7	Distance Accuracy Evaluation	22
3.5	Point Cloud Merging	24
3.5.1	System Overview and Mapper Pipeline	24
3.5.2	Registration and Supporting Modules	25
3.5.3	Surface Fusion Module	26
3.6	2D Map Generation and Cost Modeling	28
3.6.1	Costing computation.	29
3.6.2	Environment types and state conversion.	30
3.6.3	Map overwrite and merging.	30
3.7	Path Planning	31
3.7.1	Path-Planning Overview	31
3.7.2	Perception and Map Update	32
3.7.3	Path Planning and Decision Making	32
3.7.4	Motion and Constraints	32
3.7.5	Multi-Robot Coordination	33
3.7.6	Design Justification and Comparison with Existing Methods	33
4	Results, analysis, and discussion	35
4.1	Scanning Result Analysis	35
4.2	Time Consumption and Pipeline Analysis	36
4.3	Single-Robot Scene Scanning Experiment	37
4.4	Multiple Vehicles Scanning Simultaneously	39
5	Conclusions and Future Work	40
5.1	Conclusion	40
5.2	Future Work	41
6	Reference	41
7	Contribution	43
7.1	Roles of Team Members in the Project	43
7.2	Subsections and Responsible Authors	44
A	Hardware Iteration of the Robot Platform	46
A.1	First-Generation Robot	46
A.2	Comparison Between the Two Generations	48
B	Image-to-Point-Cloud Reconstruction Appendix	48
B.1	Image-to-Point-Cloud Reconstruction	48
C	Feature Coarse Registration	48

D	Experimental Point-Cloud-to-2D Map Techniques	49
D.1	Theoretical, Mathematical and Scientific Background	49
D.1.1	Theoretical Basis of 2D Grid-Based Terrain Representation	49
D.1.2	Mathematical Definitions of Terrain Geometric Features	49
D.1.3	Theoretical Basis of Semantic Traversability Classification	50
D.1.4	Mathematical Criteria for Hard Blocking	50
D.1.5	Theoretical Basis of Continuous Traversability Cost	51
D.1.6	Rationale for Separating Hard Block and Soft Cost	51
D.2	Methods, Setups and Software Implementation	52
D.2.1	Implementation Pipeline	52
D.2.2	Point-Cloud Preprocessing and Grid Representation	52
D.2.3	Ground-Layer Analysis and Semantic Map Generation	52
D.2.4	Cost-Map Generation	53
D.2.5	Representative Map Outputs and Implementation Evaluation	53
D.2.6	Quantitative Comparison and Design Analysis	55
D.2.7	Discussion of Robustness and Limitations	56
E	Repositories Related to This Work	57
F	AI Assistance Statement	58

This project investigates a low-cost multi-robot indoor scanning system based on monocular vision and centralized computation. The aim is to reduce the hardware cost and physical complexity of conventional SLAM-style systems while still producing a usable pipeline for environment reconstruction, map generation, and autonomous exploration. The proposed framework combines compact mobile robots for image capture and coarse pose estimation with a host computer that performs image-to-point-cloud reconstruction, point cloud merging, 2D map generation, and path planning.

The work integrates several modules into a complete end-to-end system. A monocular depth model is used to reconstruct coloured point clouds from single RGB images. These local point clouds are then aligned and fused into a larger global representation. The fused geometry is further converted into a 2D traversability map that distinguishes blocked, traversable, and unknown regions, and a frontier-based cost-aware path-planning method is used to support both single-robot and multi-robot exploration. In addition, the project includes the design and implementation of a compact low-cost robot platform suitable for distributed data acquisition.

Experimental results show that the system can recover the approximate spatial relationships of major objects in indoor scenes and can support map-based exploration without relying on expensive sensing hardware such as LiDAR. The results also show that pipeline parallelism improves throughput in the multi-robot setting. However, the system still exhibits several limitations, including geometric inaccuracy in monocular reconstruction, point cloud fragmentation, false obstacle generation, single-layer 2D map limitations, and coordination overhead in dense multi-robot scenarios.

Overall, the project demonstrates the feasibility of combining monocular reconstruction, point cloud fusion, map generation, and cooperative path planning in a low-cost framework. The results suggest that future improvement should focus on reconstruction consistency, scalable map representation, and tighter coupling between perception, fusion, and planning.

1 Introduction and Literature Review

Recent advances in robotics and embedded computing have made autonomous mapping and exploration increasingly relevant to low-cost indoor platforms. However, in practical deployment, the challenge is not only to sense the surrounding geometry, but also to convert raw observations into a representation that can support navigation, task allocation, and path planning. In other words, a useful mapping system is not a single isolated algorithm, but a complete pipeline that connects image acquisition, geometric reconstruction, map fusion, traversability analysis, and decision making.

For many real applications, such as rapid indoor scanning, low-cost inspection, and collaborative exploration in cluttered environments, mainstream SLAM solutions remain difficult to scale directly. High-end systems often rely on expensive sensors, large robot bodies, or significant computational resources. These design choices can improve accuracy, but they also increase cost, system complexity, and deployment difficulty. This creates a clear practical gap between highly capable research or industrial platforms and compact systems that can be deployed cheaply in larger numbers.

This project addresses that gap by investigating a low-cost multi-robot scanning framework based on monocular vision and centralized processing. The core idea is to use multiple small robots as distributed data-acquisition units, while the host computer performs the computationally expensive stages of image-to-point-cloud reconstruction, point cloud merging, 2D map generation, and path planning. The project is therefore motivated by the following question: to what extent can a low-cost monocular multi-robot system provide useful environment reconstruction and exploration performance without relying on expensive sensing hardware?

More specifically, the goals of the project are to:

- design and validate a compact, low-cost robot platform for image capture, communication, motion, and coarse pose estimation;
- reconstruct usable point clouds from monocular images and merge them into a larger global representation;
- convert the fused point cloud into a 2D map suitable for traversability reasoning and path planning;
- evaluate whether a multi-robot centralized architecture can improve scanning efficiency while keeping the per-robot hardware cost low.

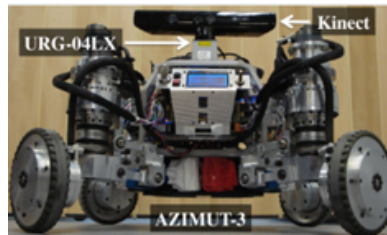
1.1 Related Work on Low-Cost SLAM and Multi-Robot Exploration

Recent advances in SLAM technologies have led to a wide range of systems for indoor navigation, mapping, and inspection. Mature frameworks such as ORB-SLAM2, RTAB-Map, and Cartographer demonstrate that strong localization and mapping performance can be achieved through tightly engineered sensing and fusion pipelines [1–3]. These systems show the value of combining perception, geometric modeling, and iterative correction, and they provide an important reference point for this project.

The literature reveals a clear tradeoff. Multi-sensor or LiDAR-dominant systems usually provide stronger precision, robustness, and engineering maturity, but these benefits come at the cost of higher hardware expense, greater physical size, and more difficult system integration. This is especially important in scenarios where many robots are needed,



(a) Clearpath Husky UGV



(b) Intel RealSense D435i + RTAB-Map



(c) Boston Dynamics Spot

Figure 1: Representative SLAM-based robotic platforms discussed in the literature review.

where scanning must be performed quickly, or where the environment is narrow and cluttered. In such cases, system scalability becomes not only a perception problem but also a cost and deployment problem.

At the same time, lower-cost visual methods have become increasingly attractive because they reduce hardware requirements and can provide rich scene information. Recent monocular depth and visual reconstruction methods suggest that useful 3D structure can be inferred from far simpler sensing hardware than traditional LiDAR-based systems require. However, the literature also indicates several shortcomings that remain directly relevant to this project: monocular reconstruction is less stable geometrically, visual methods are sensitive to scene conditions, and the consistency needed for later fusion and mapping is harder to guarantee. In addition, many existing systems focus either on local reconstruction quality or on exploration strategy, without fully addressing how low-cost perception, map fusion, and multi-robot planning interact in one complete system.

Research on multi-robot exploration further informs this work. Multi-robot systems can improve environmental coverage and reduce total exploration time by parallelizing data collection, but they also introduce coordination costs, conflict handling, redundant coverage, and map-sharing challenges. This means that adding more robots does not automatically translate into proportional gains in efficiency. The literature therefore suggests that the key challenge is not simply to build a low-cost robot or to deploy multiple robots, but to design a system in which perception quality, fusion stability, and planning decisions remain compatible across the whole pipeline.

These observations directly inform the goals of the present project. Rather than trying to compete with high-end SLAM systems in absolute accuracy, this work focuses on a different gap in the literature: how to build a practical, scalable, and low-cost scanning pipeline that remains useful despite weaker sensing hardware. The project therefore combines monocular perception, centralized computation, swarm-style exploration, and map-based planning in order to test whether acceptable reconstruction and exploration performance can still be achieved under stronger cost constraints.

2 Theoretical, Mathematical and Scientific Background

2.1 Relative Pose Estimation of the Robot

2.1.1 Coordinate Systems and the Chain Rule

To estimate the relative position of the robot, multiple reference frames are introduced. In this work, the fixed world reference frame is denoted as the global coordinate system $\{W\}$, while the coordinate system attached to the robot body is denoted as the local coordinate system $\{B\}$. The global coordinate system provides a unified reference for describing the robot pose in the environment, whereas the local coordinate system is more suitable for representing short-term motion increments.

The pose of the robot in the global coordinate system can be written as the homogeneous transformation matrix

$${}^W T_B = \begin{bmatrix} {}^W R_B & {}^W p_B \\ 0 & 1 \end{bmatrix}$$

where ${}^W R_B$ is the rotation matrix of the robot body frame relative to the global frame, and ${}^W p_B$ is the position vector of the robot origin in the global frame. If a point P is represented in the local coordinate system as ${}^B p$, then its coordinates in the global coordinate system can be expressed as

$${}^W p = {}^W R_B {}^B p + {}^W p_B.$$

This equation shows that a point is first rotated and then translated when transformed from the local frame to the global frame.

To update the pose across consecutive time steps, the chain rule of coordinate transformation is adopted. Let the robot body frames at time $k-1$ and k be denoted by $\{B_{k-1}\}$ and $\{B_k\}$, respectively. Their relationship can be written as

$${}^W T_{B_k} = {}^W T_{B_{k-1}} {}^{B_{k-1}} T_{B_k}.$$

Here, ${}^{B_{k-1}} T_{B_k}$ represents the local pose increment between two adjacent time steps. Based on this chained relationship, the robot pose can be recursively updated in the global frame, or equivalently represented relative to any chosen reference frame.

2.1.2 Coordinate Transformation Update Based on Velocity and Attitude

When the local displacement increment and the previous attitude of the robot are known, the current coordinate transformation can be updated recursively. Let Δp_{k-1} denote the displacement increment in the local frame at time $k-1$. The robot position in the global coordinate system at time k can then be computed as

$$p_k = p_{k-1} + R_{k-1} \Delta p_{k-1},$$

where p_{k-1} is the global position vector at the previous time step, and R_{k-1} is the rotation matrix of the robot body frame relative to the global frame. This equation indicates that the local motion increment must first be rotated into the global frame before being added to the previous position.

The local displacement increment can be approximated from the robot velocity and the sampling interval. If the forward linear velocity is v and the sampling interval is Δt , then

$$\Delta p_{k-1} = \begin{bmatrix} v \Delta t \\ 0 \\ 0 \end{bmatrix}.$$

This formulation assumes that the robot motion within one time step mainly occurs along the forward axis of the body frame. In practice, the robot heading may change continuously during motion. Therefore, a smaller Δt usually leads to better consistency between the assumed motion direction and the actual attitude, which improves the accuracy of the estimated position.

Using this recursive formulation together with the chain rule, the robot pose can be expressed not only in the global frame but also relative to any selected reference frame. However, since this method depends on the commanded velocity, the sampling interval, and the attitude measurement result, the estimation error accumulates over time. Therefore, external position feedback or other correction information should be introduced periodically to suppress long-term drift.

2.2 Image-to-Point Cloud Reconstruction

2.2.1 Monocular Depth Estimation

This work is based on monocular depth estimation and camera projection geometry. The aim of this submodule is to recover a usable point cloud from a single RGB image. This is important because the point cloud is later used for fusion, mapping, and path planning in the full team system.

Monocular depth estimation predicts scene depth from one image [4]. A single RGB image only records the 2D appearance of a scene. It contains colour, brightness, texture, edges, and object shapes, but it does not directly provide real 3D distance. In other words, the image shows how the scene looks on the image plane, not how far each point is from the camera in physical space. The depth of each pixel must therefore be inferred from visual cues such as perspective, occlusion, scale, texture variation, and object structure. For example, objects that appear smaller may be farther away, and strong perspective lines may suggest depth change across a surface. However, these cues are indirect, so the problem is difficult and often ambiguous. Different real 3D scenes can produce similar 2D image patterns. Even so, monocular depth estimation is attractive for low-cost robotic systems because it only requires one camera. This reduces hardware cost, weight, and system complexity.

2.2.2 Relative Depth and Metric Depth

A key idea in this area is the difference between relative depth and metric depth. Relative depth mainly describes the order of depth in a scene. It can show which parts are closer and which parts are farther away, but the numerical values do not necessarily match real physical distance. Metric depth, by contrast, tries to produce values that are more consistent with true scene geometry. Recent monocular depth models such as Depth Anything V2 and Depth Pro reflect this broader move toward stronger generalisation and more practical geometric output [5,6]. For this project, this distinction is important because the depth map is not the final result. It must be converted into a point cloud and then passed to later modules. If the depth is only correct in a relative sense, the reconstructed point cloud may still have incorrect scale and distance even if the shape looks visually reasonable. This would reduce its value for later geometric processing such as point cloud fusion and map construction.

2.2.3 Pinhole Camera Model

The conversion from a depth map to a point cloud is based on the pinhole camera model [7]. This model describes how a 3D scene is projected onto a 2D image plane through a camera. In this model, each image pixel corresponds to a ray that starts at the camera centre and passes through the image plane. If the depth value of a pixel is known, the 3D position of the corresponding scene point can be recovered along that ray. This is why depth estimation and camera geometry must be considered together. The basic geometry of the pinhole camera model is illustrated in Fig. 2. The camera intrinsics define how image coordinates relate to directions in 3D space. These intrinsics include the focal lengths in the horizontal and vertical directions, denoted by f_x and f_y , and the principal point, denoted by (c_x, c_y) , which represents the optical centre of the image.

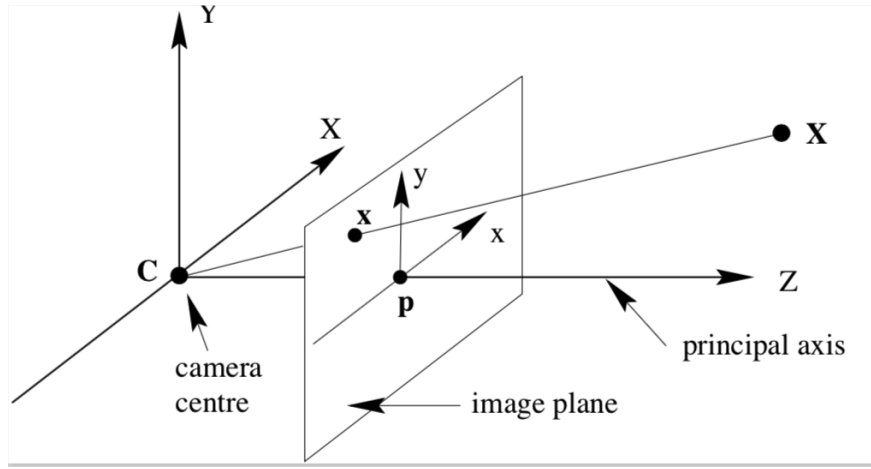


Figure 2: Geometry of the pinhole camera model.

2.2.4 Back-Projection Geometry

Let (u, v) be the image coordinates of a pixel and let Z be its depth. If the focal lengths are f_x and f_y , and the principal point is (c_x, c_y) , the corresponding 3D point (X, Y, Z) in the camera coordinate system can be written as

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}, \quad Z = Z.$$

These equations describe the back-projection process. The term $(u - c_x)$ gives the horizontal offset of the pixel from the optical centre, and $(v - c_y)$ gives the vertical offset. Dividing by focal length converts these image offsets into normalised camera coordinates. Multiplying by Z then scales the point to its position in 3D space. In this way, the depth value controls how far the point lies from the camera, while the pixel position determines its lateral location. This back-projection step is the main mathematical link between the depth map and the final point cloud.

2.2.5 Camera Calibration

The quality of the reconstructed point cloud also depends on camera calibration [8]. A real camera does not perfectly follow the ideal pinhole model. Lens distortion changes the relationship between image position and the true viewing ray. If the intrinsic parameters

are inaccurate, or if distortion is not handled properly, the reconstructed 3D points will contain systematic geometric error. This means that even if the depth prediction is reasonable, the final point cloud may still show incorrect shape, position, or scale. For this reason, calibration is important for improving geometric consistency in single-image point cloud reconstruction.

2.3 Point Cloud Merging

2.3.1 Registration

Due to technical limitations, robotic control systems are unable to provide perfectly accurate initial poses for point clouds. Therefore, a registration step is required before merging individual point clouds. Point cloud registration refers to estimating a spatial transformation that aligns multiple point clouds into a common reference coordinate system. The

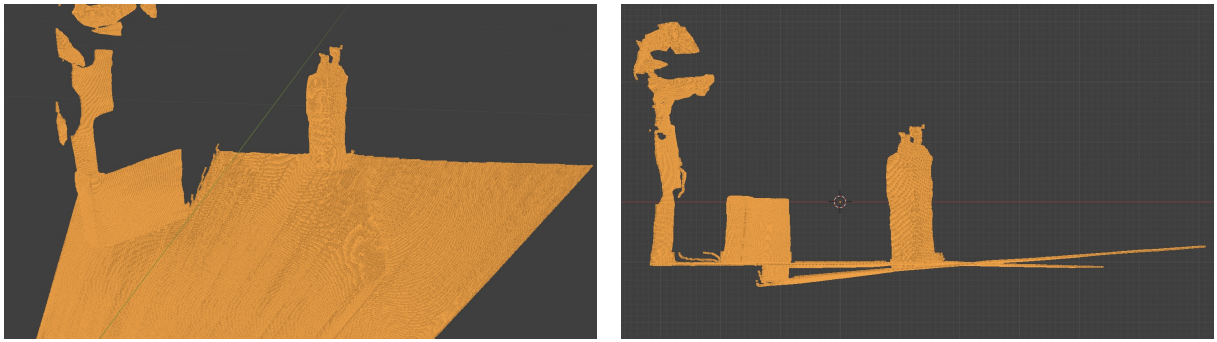


Figure 3: Merged result only based on initial pose.

most fundamental method for point cloud registration is the Iterative Closest Point (ICP) algorithm [9]. The core idea of ICP is to iteratively perform nearest-neighbour correspondence matching and rigid transformation estimation, gradually minimizing the alignment error between two point clouds. Given a source point cloud $\mathcal{P}$ and a target point cloud $\mathcal{Q}$, ICP establishes correspondences and minimizes the following objective function:

$$E(R, t) = \sum_i \|q_i - (Rp_i + t)\|^2 \quad (1)$$

where R and t represent the rotation and translation, respectively. This problem is typically solved efficiently using Singular Value Decomposition (SVD), and the algorithm converges iteratively to a local optimum.

In this project, the input point cloud is reconstructed from monocular images using an AI model. The primary errors in such point clouds include depth noise along the viewing direction and local geometric distortions, such as scale drift and structural stretching. Under these conditions, ICP becomes unsuitable because it implicitly assumes isotropic Gaussian noise, that is, the covariance satisfies $\Sigma = \sigma^2 I$. However, in monocular reconstruction, uncertainty along the depth direction is significantly larger than in the tangential directions, resulting in strongly anisotropic error distributions. Consequently, ICP assigns excessive weight to depth errors, leading to systematic bias in the optimization results.

In contrast, the Generalized Iterative Closest Point (GICP) algorithm [10] introduces local covariance modelling to address this issue. GICP represents each point as a Gaus-

sian distribution incorporating local geometric structure and minimizes the Mahalanobis distance:

$$E(R, t) = \sum_i (q_i - (Rp_i + t))^T (C_i^p + RC_i^q R^T)^{-1} (q_i - (Rp_i + t)) \quad (2)$$

This formulation allows deviations within local geometric structures without being penalized as large errors. Therefore, GICP avoids over-amplifying depth noise and significantly improves both stability and accuracy of registration in this project.

2.3.2 Surface Fusion

When constructing a 3D map from multiple frames of point clouds, simply stacking registered point clouds often leads to noise accumulation and surface discontinuities. Therefore, a robust surface representation method is required to consistently integrate multiple observations. In this project, a surfel-based representation [11] is adopted. A surfel (surface

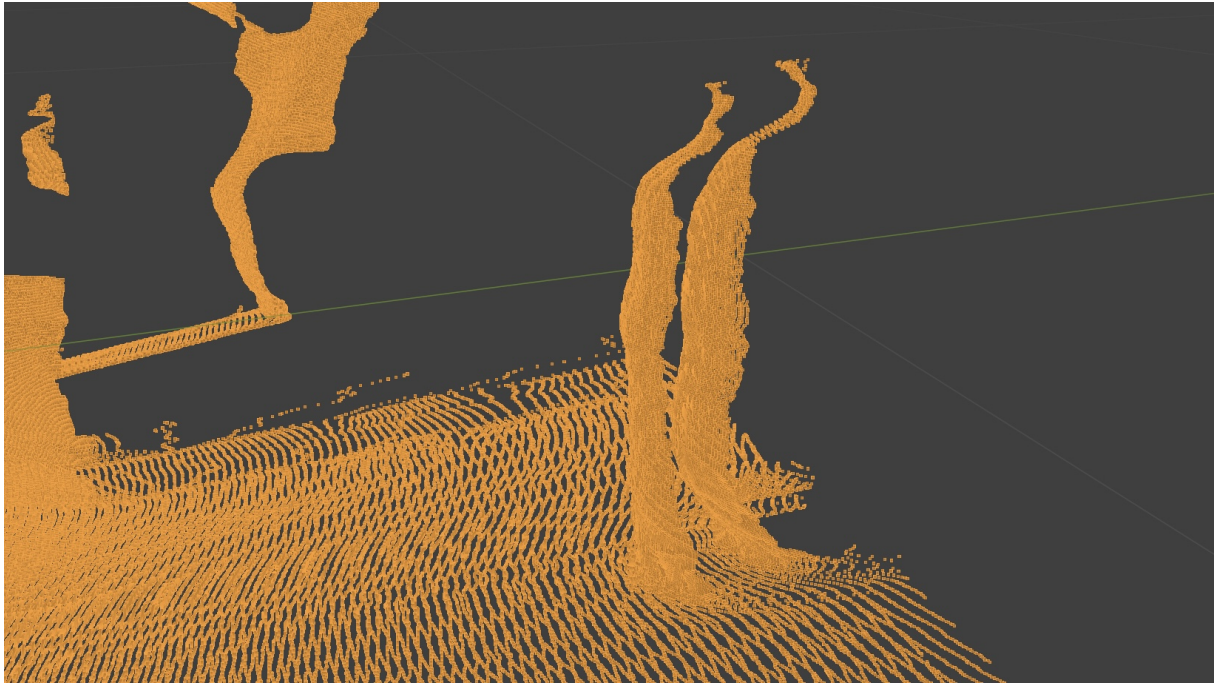


Figure 4: Merged result without surface fusion.

element) approximates a local surface region and typically contains attributes such as position, normal vector, colour, and a confidence weight representing observation reliability. Compared to discrete point representations, surfels incorporate local geometric structure, enabling progressive refinement of surface estimates across multiple observations and improving smoothness and consistency.

Surface fusion can be formulated as an incremental update problem. For each new frame, correspondences between observations and existing surfels must be established, followed by updating surfel attributes. Data association is typically based on geometric and photometric consistency measures, such as position, normal direction, and colour similarity.

For matched observations, the fusion process usually follows a weighted update scheme:

$$A_{\text{new}} = \frac{w_{\text{old}}A_{\text{old}} + w_{\text{obs}}A_{\text{obs}}}{w_{\text{old}} + w_{\text{obs}}} \quad (3)$$

where A denotes a surfel attribute, such as position or color, and w represents the confidence weight. This formulation can be interpreted as a recursive estimation process, allowing surfel attributes to converge to stable values over multiple observations. For observations that cannot be matched to existing surfels, new surfels are introduced to expand the map representation.

Additionally, maintenance operations are required to ensure long-term map stability, such as merging redundant surfels and removing unreliable ones.

Overall, surfel-based surface fusion effectively integrates noisy multi-frame observations while preserving local geometric structure, making it particularly suitable for scenarios with unstable input quality.

2.4 2D Map Generation and Cost Modeling

2.4.1 Two-Dimensional Grid Map and Its Role

To use a 3D point cloud for subsequent route search and path planning, the environment is first discretized into a two-dimensional grid map [12]. The space is divided into regular cells with a fixed resolution, and each cell represents a local region of the environment. In this way, the originally continuous and large-scale point cloud is compressed into a regular discrete representation.

The main role of the 2D grid map is to provide a discretized search space for path-planning algorithms. Direct search on the raw point cloud is computationally expensive because the data scale is large and the adjacency relationship among points is complicated. After grid discretization, path search only needs to be performed among a finite number of cells, which makes the neighborhood relationship clearer and reduces the computational burden [12, 13].

In addition, the 2D grid map helps control the growth of complexity as the map becomes larger. Local geometric information in each region can be summarized by a traversal cost $cost$, where a larger value indicates more difficult traversal and a smaller value indicates better traversability. In this way, the environment is represented as a collection of cost-associated cells, which provides a compact and efficient input for subsequent path planning.

2.4.2 Ground Layer Selection

During the construction of a two-dimensional grid map, a single grid cell may contain more than one layer of points. In addition to the actual ground, points may also come from the bottom of obstacles, suspended structures, or noise. Therefore, the lowest point in a cell cannot be directly regarded as the ground. To address this issue, candidate layers are first constructed along the vertical direction in each grid cell, and then the layer that most likely represents the actual ground is selected [14].

Let the candidate layer set of the i -th grid cell be

$$\mathcal{L}_i = \{L_{i,1}, L_{i,2}, \dots, L_{i,m_i}\}$$

where m_i is the number of candidate layers in the cell. For each candidate layer $L_{i,k}$, its

representative height is defined as

$$h_{i,k} = \frac{1}{N_{i,k}} \sum_{p \in L_{i,k}} y(p)$$

where $N_{i,k}$ is the number of points in $L_{i,k}$, and $y(p)$ is the height coordinate of point p .

Since the actual ground is usually locally continuous, ground-layer selection should depend not only on the height distribution inside the current cell but also on the ground height of neighboring cells [13, 14]. Let $\hat{h}_{\mathcal{N}(i)}$ denote the reference ground height in the neighborhood of the i -th cell. Then the ground layer of the current cell can be defined as

$$L_i^* = \arg \min_{L_{i,k} \in \mathcal{L}_i} |h_{i,k} - \hat{h}_{\mathcal{N}(i)}|$$

which means that the candidate layer whose height is closest to the neighborhood reference ground height is selected as the ground layer. To avoid incorrectly classifying a layer with a large height difference as ground, an additional continuity constraint can be introduced:

$$|h(L_i^*) - \hat{h}_{\mathcal{N}(i)}| < \tau_h$$

where τ_h is the ground-continuity threshold. This candidate-layer selection strategy based on neighborhood continuity can extract the actual ground more robustly and reduce the interference caused by suspended structures, obstacle boundaries, and local noise.

2.4.3 Eigenvalues and Eigenvectors for Geometric Characteristics

After ground-layer selection, the local geometric characteristics of each grid cell still need to be analyzed in order to evaluate traversability. In this work, eigen-analysis is not performed on the entire map as a whole. Instead, it is carried out separately for each grid cell by constructing a covariance matrix from its local point set [15]. As a result, the obtained eigenvalues and eigenvectors describe the local surface properties of each individual cell rather than the global properties of the entire map.

For the i -th grid cell, let its local point set be

$$\mathcal{P}_i = \{p_1, p_2, \dots, p_{N_i}\}, \quad p_k \in \mathbb{R}^3$$

and let its centroid be

$$\bar{p}_i = \frac{1}{N_i} \sum_{k=1}^{N_i} p_k$$

Then the covariance matrix of this cell is defined as

$$C_i = \frac{1}{N_i} \sum_{k=1}^{N_i} (p_k - \bar{p}_i)(p_k - \bar{p}_i)^T$$

The eigen-decomposition of C_i is written as

$$C_i n_{i,j} = \lambda_{i,j} n_{i,j}, \quad j = 1, 2, 3$$

with

$$\lambda_{i,1} \leq \lambda_{i,2} \leq \lambda_{i,3}$$

If the local point set in the cell is approximately distributed on a plane, then the eigenvector corresponding to the smallest eigenvalue, $n_{i,1}$, can be regarded as the approximate normal vector of the local ground surface [15]. Let the unit vertical direction be

$$v = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

This eigenvector can also be treated as the surface orientation vector for slope evaluation. Then the local slope of the cell can be expressed as

$$\theta_i = \arccos \left(\frac{n_{i,1} \cdot v}{\|n_{i,1}\| \|v\|} \right)$$

where a larger θ_i indicates a larger local surface inclination.

At the same time, the smallest eigenvalue $\lambda_{i,1}$ reflects the dispersion of points along the normal direction and can therefore be used to characterize the local roughness or unevenness of the cell. A normalized roughness index can be defined as

$$r_i = \frac{\lambda_{i,1}}{\lambda_{i,1} + \lambda_{i,2} + \lambda_{i,3}}$$

where a larger r_i indicates a rougher or less stable local surface.

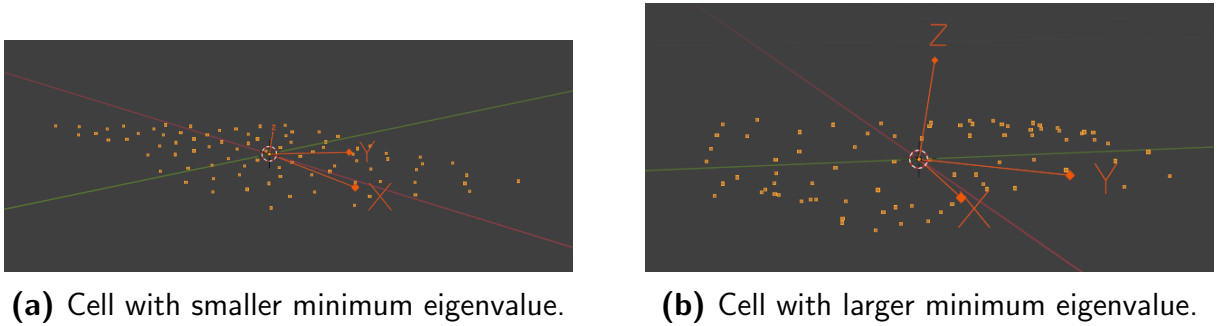


Figure 5: cells with different minimum eigenvalues.

Therefore, the eigenvector associated with the smallest eigenvalue mainly describes the local directional property of each grid cell, while the smallest eigenvalue itself reflects the stability of the cell along the surface normal direction. By analyzing slope and roughness on a cell-by-cell basis, the local traversability of different regions can be quantified and used for subsequent cost-map construction. As illustrated in Fig. 5, two cells may exhibit similar slope angles while still having different roughness levels. The annotated z -vector in each figure corresponds to the eigenvector associated with the smallest eigenvalue, and the angle between this vector and the world z -axis represents the local slope, which is approximately 17° in both cases. Compared with Fig. 5a, the cell in Fig. 5b contains a local pit and is therefore geometrically more irregular, leading to a larger minimum eigenvalue.

2.5 Path Planning

2.5.1 Map Representation and Frontier Definition

This project uses a two-dimensional occupancy grid to represent the environment:

$$m_{i,j} \in \{-1, 0, 1\}$$

where -1 denotes UNKNOWN, 0 denotes FREE, and 1 denotes OCCUPIED. On this basis, a frontier is defined as

$$Frontier = \{(i, j) \mid m_{i,j} = FREE, \exists neighbour = UNKNOWN\}.$$

That is, a frontier cell is traversable itself and is adjacent to at least one unknown cell.

2.5.2 Robot Motion and Coordinate Model

The robot moves in continuous space, and its state is represented as

$$s = (x, z, \theta)$$

where θ is the orientation angle. The motion direction is determined by

$$\Delta r = \cos(\theta), \quad \Delta c = \sin(\theta),$$

and the update rule is

$$r_{t+1} = r_t + \Delta r \cdot d, \quad c_{t+1} = c_t + \Delta c \cdot d,$$

where d represents the distance moved.

2.5.3 Path Planning Method (A^*)

Path planning uses the A^* algorithm with

$$f(n) = g(n) + h(n), \quad h(n) = |x_n - x_g| + |y_n - y_g|.$$

To incorporate environmental risk into the path cost, the cost function is extended as

$$g(n) = g(n-1) + 1 + \lambda \cdot cost(n).$$

This allows the planner to prefer shorter paths while also avoiding high-risk areas.

2.5.4 Collision Detection Model

The robot is modeled as a circle whose feasible region satisfies

$$(x - x_0)^2 + (y - y_0)^2 \leq r^2$$

where r is the robot radius. During pathfinding, the path is sampled continuously, and all sampled points must remain in traversable space for the path to be considered safe.

2.5.5 Perception Models and Information Acquisition

The robot uses an FOV model for perception:

$$\theta \in \left[\theta_0 - \frac{FOV}{2}, \theta_0 + \frac{FOV}{2} \right].$$

The number of visible unknown cells within the field of view is

$$U = \sum 1_{cell=UNKNOWN}.$$

This helps guide the robot toward directions with greater potential information gain.

2.5.6 Observation Overlap Constraint

To ensure adjacent observations can be fused, an overlap constraint is introduced:

$$Overlap = \frac{FOV - |\Delta\theta|}{FOV}.$$

To preserve sufficient overlap between neighboring observations, the planner enforces

$$Overlap \geq min_overlap_ratio.$$

2.5.7 Single-Robot Decision-Making Model

The robot selects targets using a composite scoring function:

$$Score = Path_{length} + Yaw_{penalty} + Cost_{penalty} + Overlap_{penalty} - Unknown_{gain}.$$

Here, path length, turning cost, and environmental risk are treated as penalties, while newly observable unknown area is treated as gain. The robot selects the target with the lowest score.

2.5.8 Multi-Robot Collision Avoidance

In the multi-robot setting, collision avoidance is enforced through position and distance constraints:

$$(x_i, y_i) \neq (x_j, y_j), \quad d(r_i, r_j) > d_{min}.$$

2.5.9 Field-of-View Constraints and Cooperative Avoidance

When one robot may enter the field of view of another,

$$r_j \in FOV_i,$$

the corresponding area is treated as forbidden during planning:

$$M_{plan} = M \setminus Forbidden.$$

This helps reduce visual interference and improves cooperative exploration stability.

3 Methods, Setups and Software Implementation

3.1 Architecture Overview

This work adopts a swarm-based scanning strategy, in which multiple robots scan the environment in parallel. Compared with a single-robot solution that scans the scene region by region, this approach can collect environmental information over a larger area within a shorter time. Meanwhile, the number of deployed robots can be adjusted according to the environment size and practical constraints such as time and cost, which makes the overall scheme more flexible.

Since AI-based models are used in the subsequent processing stages, the system requires relatively high computing power. However, if every individual SLAM robot were equipped with a high-performance NPU or other dedicated AI hardware, the total hardware cost in a multi-robot scenario would increase significantly, as shown in Fig. 6. To reduce this cost, a centralized data-processing architecture is adopted. Under this architecture, all SLAM robots are treated as executors and sensors. Their onboard functions are limited to motion, image acquisition, and pose-related data collection, while computationally intensive

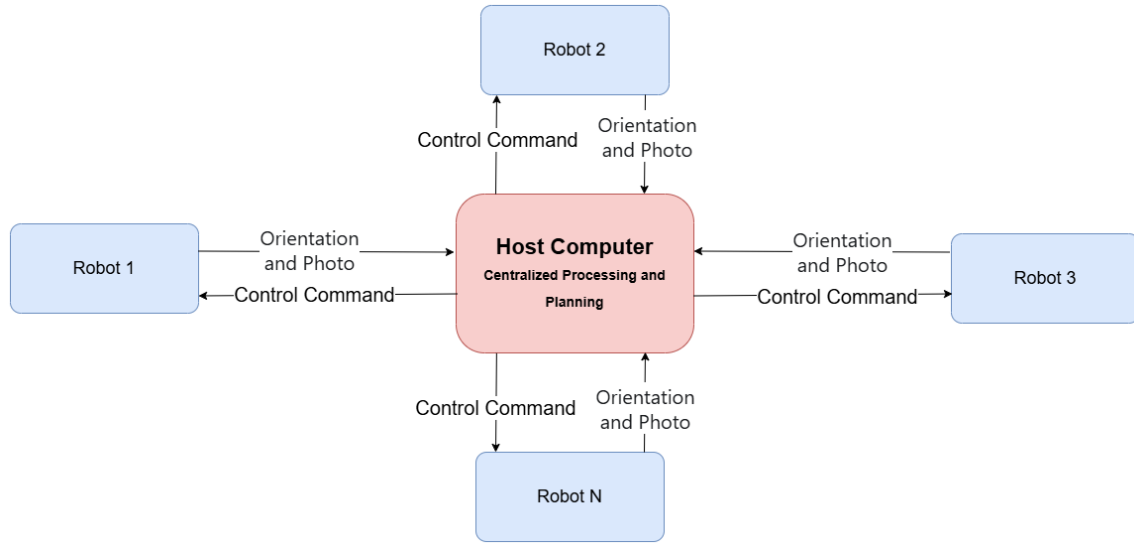


Figure 6: Centralized processing architecture for swarm monocular SLAM robots.

AI inference is not performed locally. Instead, all collected data are transmitted through the network to a high-performance host computer, which is responsible for unified data processing, map construction, and path planning. In this way, the high computational load is concentrated on a single terminal, effectively reducing the hardware cost of each robot.

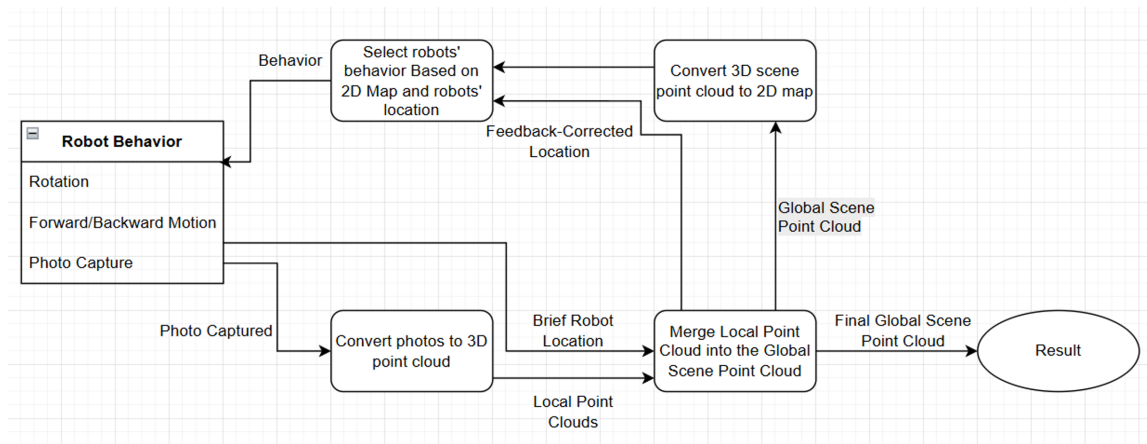


Figure 7: Overview flow chart of single-robot control and data acquisition.

The overall operation flow is shown in Fig. 7. After the host computer receives the images and coarse pose estimates uploaded by the robots, it first applies Depth Pro to convert the images into depth maps, and then reconstructs the point cloud corresponding to each image. Based on the coarse pose estimates, the host computer performs post-processing on each point cloud and merges it into the global point cloud. At the same time, the current robot pose can be further refined using the feedback from the merging error, so that a more accurate pose estimate is obtained. On this basis, the global point cloud is further converted into a 2D map, which is used to determine traversable regions, blocked regions, unexplored regions, and the traversal cost of different areas. The path-

planning module then generates paths and motion commands for each robot according to the refined poses and the current 2D map. When all robots have completed the scanning task, that is, when no further valid path can be planned, the scanning process ends. Finally, the merged global point cloud is exported as the output of the complete scanning task.

3.2 Robot Platform

The robot hardware used in this project was developed through two design iterations. The first-generation platform was built to validate the overall feasibility of monocular image capture, motion control, and pose-related feedback on a compact mobile platform. Based on the practical limitations observed in that prototype, a second-generation robot was then developed and adopted as the final hardware platform of the system. The final platform is a compact and highly integrated robot with a size of approximately $10 \times 10 \times 10$ cm, and includes image acquisition, wireless communication, motion execution, and coarse pose estimation capabilities. It uses an ESP32-CAM controller, an OV3660 camera, an MPU6050 IMU, and a motor-driving structure based on L298N and N15 geared DC motors [16–18]. This design preserves the minimum hardware functions required by the system while keeping the robot small and low-cost.

3.2.1 Second-Generation Platform

To address the limitations of the first-generation platform in size, power consumption, stability, and pose estimation accuracy, a second-generation robot was developed and finally adopted in this work. The size was reduced to approximately $10 \times 10 \times 10$ cm, which better satisfied the project requirements for miniaturization and low cost. The detailed hardware design of the first-generation robot is provided in Appendix A. The second-generation robot adopted ESP32-CAM as the main controller. This chip provides relatively strong performance in a compact form factor while also integrating Wi-Fi communication and a camera interface, which makes it well suited for this project [16]. Although the number of available pins is limited, the hardware requirements of the system can still be satisfied by reusing several special-purpose pins.

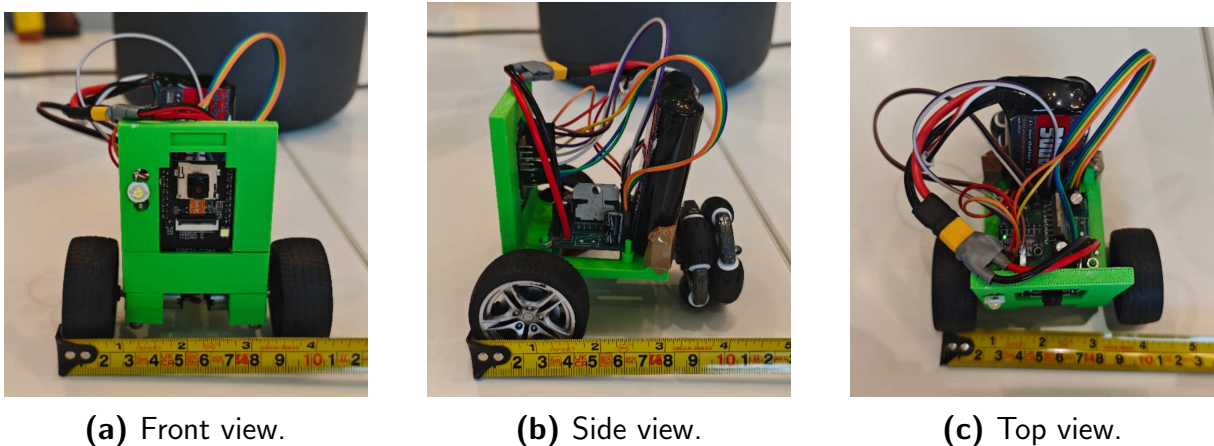


Figure 8: Three-view images of the second-generation robot platform.

In the camera module, OV3660 was selected because it supports a resolution of 1024×768 , which is close to the effective input scale of Depth Pro and therefore helps reduce

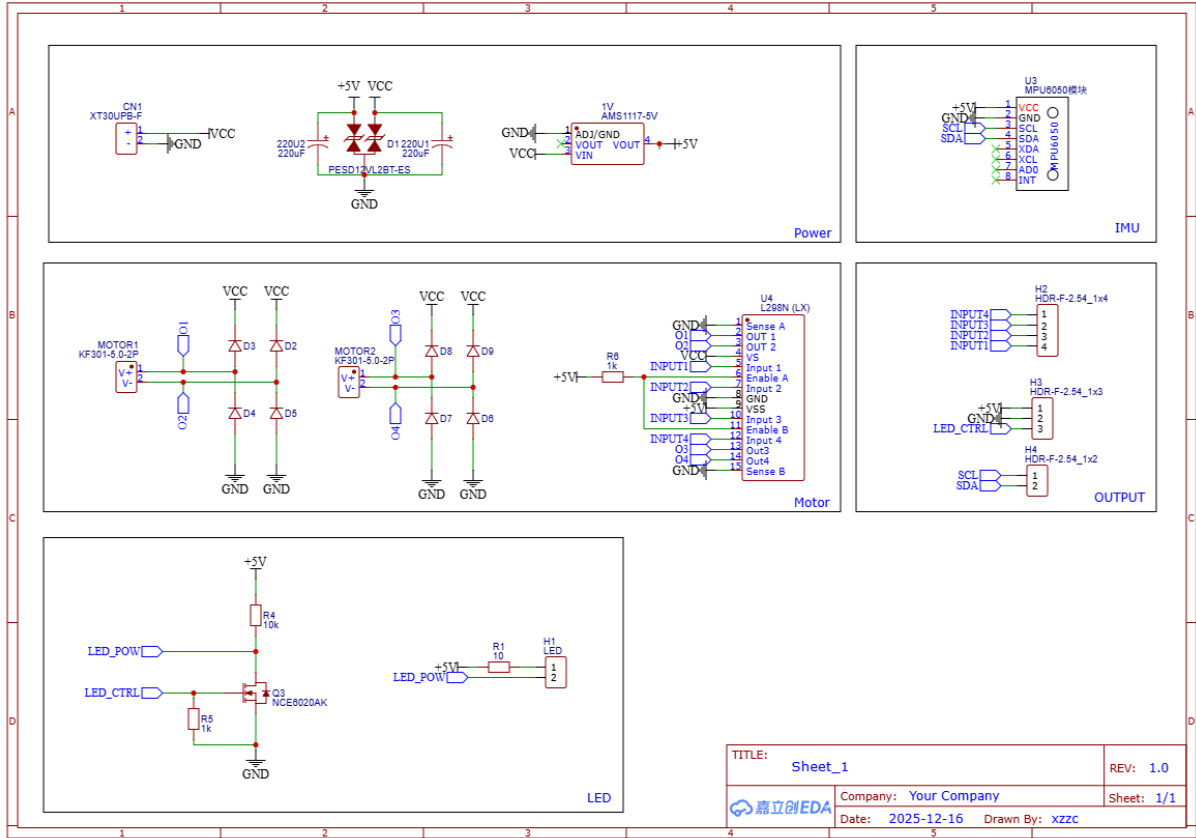


Figure 9: Circuit schematic of the second-generation robot.

unnecessary stretching loss or compression loss. Its field of view is about 45° , and lens distortion is small, which allows a relatively large scanning range without introducing strong geometric distortion. In addition, image capture is performed using the OV3660 together with LED illumination. For motion, the second-generation robot uses an L298N motor driver together with N15 DC geared motors, two rubber drive wheels, and a rear omni-wheel for support and rotation assistance [18]. Compared with the stepper-motor-based first-generation design, this arrangement provides better speed, stability, and resistance to slipping. The robot realizes forward motion, backward motion, and in-place left or right rotation through motor forward and reverse control.

For pose feedback, the second-generation robot uses an MPU6050 IMU [17]. This sensor includes a DMP chip, which reduces the computational burden of additional integration on the main controller and is also widely used and well documented. The final pose estimation strategy combines timed DC motor control with IMU-based rotation feedback. In practice, this design performed significantly better than the first-generation one, achieving a rotation error of about $\pm 3^\circ$ per 180° turn and a forward/backward error of about ± 5 mm per 10 cm of travel.

Several functions of the first-generation robot were intentionally removed in the second-generation design. Lateral translation was removed because it was rarely needed in practice and could be replaced by in-place rotation followed by forward or backward motion. The camera pitch and yaw servos were also removed, since yaw could be replaced by robot self-rotation, while pitch-view depth conversion introduced a large error of about

± 20 cm per 1 m distance. In addition, the number of LEDs was reduced because experiments showed that the effect of four dedicated fill lights was almost the same as that of one additional LED together with the built-in LED of the **ESP32-CAM**. These changes significantly reduced the overall size and structural complexity of the robot.

3.2.2 Bill of Materials and Cost Estimate

To further demonstrate the low-cost characteristic of the second-generation robot, Table 1 summarizes the main bill of materials (BOM) entries based on the exported component list. The quoted prices are taken from the JLC/LCSC marketplace [19], and all prices are given in RMB. After including the 7.4V lithium battery, the subtotal of the priced entries is RMB 93.54, which is approximately GBP 10 depending on the exchange rate.

Table 1: Main BOM entries of the second-generation robot platform.

Component	Quantity	Subtotal (RMB)
ESP32-CAM	1	46.83
7.4V lithium battery	1	33.40
L298N motor driver	1	6.19
NCE6020AK MOSFET	1	0.99
XT30UPB-F connector	1	1.29
Electrolytic capacitors	2	0.66
PESD12VL2BT-ES protection diode	1	0.32
1N4007W diodes	8	0.32
Headers and motor connectors	–	3.53
Total	–	93.54

3.3 RTOS-Based Robot Control Architecture

To improve the real-time performance and structural clarity of the onboard control software, the robot-side system was implemented using an RTOS-based architecture [20]. The whole system contains four core tasks: the WiFi feedback task, the robot control task, the photo-reading task, and the position-estimation task. These tasks cooperate with different responsibilities so that communication, motion execution, image update, and pose estimation can proceed in parallel.

The software maintains a unified global state object, denoted as **robot state**, which stores the key runtime status of the robot, including the movement mode, motor PWM, actual moving speed, latest captured image, current camera parameters, current IMU output, error logs, reference coordinate frame, and current relative pose. This state object also serves as the interface between the communication layer and the execution layer. As illustrated in Fig. 10, the left side of the architecture can be regarded as the communication layer, while the right side can be regarded as the sensing and execution layer. The communication layer does not directly read or write hardware. Instead, it only reads and modifies the shared **robot state** object according to the received packets. Conversely, the execution layer does not directly communicate with the communication layer. It continuously updates **robot state** from hardware observations and maps the relevant state attributes to hardware actions. In this way, the two parts are decoupled through the shared state object rather than directly invoking each other, which helps avoid blocking and makes the program structure clearer.

Among the four tasks, the WiFi feedback task has the highest priority. It is responsible only for communication, as well as reading and modifying the shared state object,

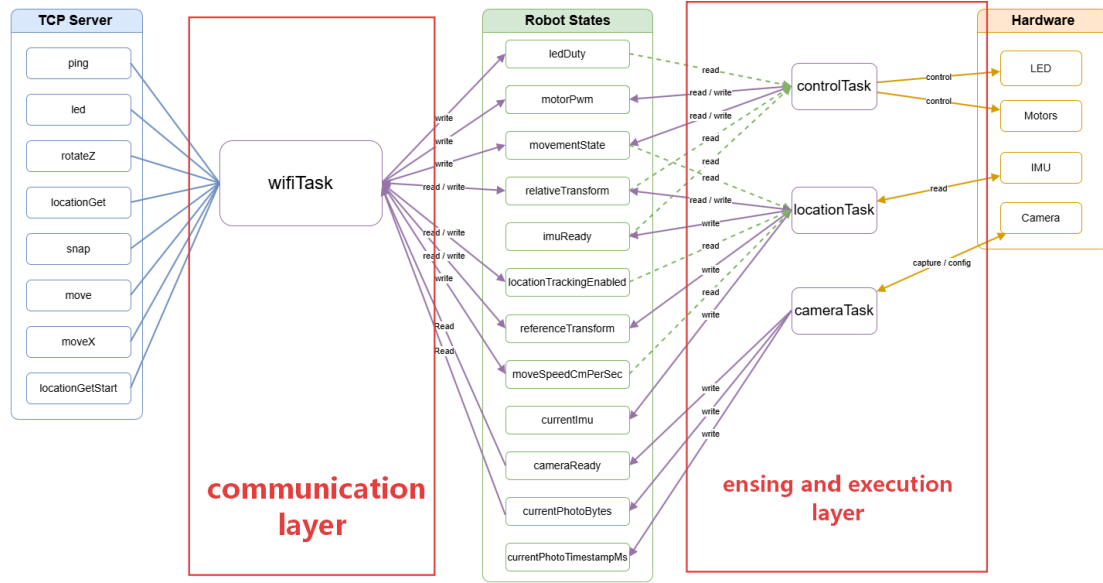


Figure 10: RTOS-based robot control architecture centered on the shared robot state object.

but it does not directly perform time-consuming operations such as IMU reading, image acquisition, or pose computation. This task remains suspended during idle periods and is awakened only when a communication packet is received. The robot control task has the second-highest priority and is also suspended most of the time. It is awakened after the WiFi feedback task updates motion-related parameters, and then updates the robot motion according to the movement mode and PWM stored in the shared state [20].

The photo-reading task and the position-estimation task have the same lower priority and run periodically in the background. The photo-reading task polls the camera at a fixed interval and continuously refreshes the latest image buffer. The position-estimation task reads the IMU periodically and updates the relative pose. By running these two tasks continuously in the background, the system avoids blocking caused by temporary hardware access when a command arrives.

To guarantee consistency in multi-task access to shared data, all shared states are protected by a mutex. In addition, any shared result must be written back to the state object only after the complete computation is finished, rather than through staged partial updates. This ensures that the WiFi feedback task always reads complete and internally consistent data when responding to external requests [20].

3.4 Image-to-Point Cloud Reconstruction

3.4.1 Pipeline Overview

This submodule converts a single RGB image into a coloured PLY point cloud. The point cloud is then passed to the next stage for point cloud fusion. After that, later stages of the project use the fused result for map generation and path planning. As the first stage of this pipeline, the main aim of the method is to produce a point cloud that is geometrically usable for later processing. An overview of the full processing pipeline is shown in Fig. 11.

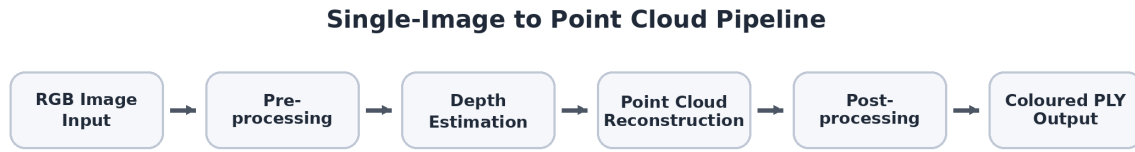


Figure 11: Overview of the single-image point cloud reconstruction pipeline.

3.4.2 Image Input and Pre-processing

The input is a single JPG image with a resolution of 1024×768 . The system processes one image at a time. A new image is captured and then processed directly. Before depth inference, the image is rotated by 180 degrees. This is done because the camera is mounted upside down during image capture. The output of the submodule is one coloured PLY file for each input image.

3.4.3 Depth Model Selection

During early development, a relative-depth model based on Depth Anything V2 was tested [5]. This model could preserve near-far order in the scene, but it was difficult to use for this project because the depth values could not be linked to real-world distance in a stable mathematical way. For this reason, the final method did not use the relative-depth approach as the main solution.

The final system uses Depth Pro for depth estimation [6]. It was selected because its metric depth output was more suitable for practical distance recovery and point cloud reconstruction in this project. After depth prediction, the depth map is converted into a 3D point cloud by back-projection with calibrated camera intrinsics. At a resolution of 1024×768 , the calibrated intrinsic parameters were $f_x = 1180.50$, $f_y = 1184.63$, $c_x = 522.11$, and $c_y = 394.13$. In the reconstruction step, Z is treated as the depth value. The X and Y coordinates are computed from pixel position and focal length. Each reconstructed point keeps its RGB colour, and the result is exported as a coloured PLY file.

3.4.4 Camera Calibration Procedure

Camera calibration was carried out using a checkerboard-based approach following the general calibration framework proposed by Zhang [8]. The intrinsic parameters were estimated from 30 checkerboard images captured from different viewing angles. The final calibration gave an OpenCV RMS value of 0.542 and a mean reprojection error of 0.459 pixels. These results were considered sufficient for the geometric reconstruction stage.

3.4.5 Point Cloud Post-processing

Several post-processing steps are applied after point cloud generation. First, the floor is extended from the nearest visible cross-section back to the origin. This is done to make the near-ground region more complete for the later mapping stage. Second, a virtual square floor patch of $12 \text{ cm} \times 12 \text{ cm}$ is added at the origin. This gives the later path-planning stage a small free region for movement. Third, only the foreground shell is kept. This

step uses depth discontinuity to detect strong depth changes and remove the transition band between foreground objects and background structure.

A representative depth map is shown in Fig. 12. In this example, the front box appears as a clear depth region against the surrounding surface. The shell-preservation step uses the depth jump around the box boundary to identify the foreground region and suppress the transition points near the object edge. This helps reduce trailing scattered points and keeps the object outline clearer for later fusion. The visual effect of this post-processing is shown in Fig. 13. Compared with the original output, the processed result contains a cleaner object boundary and fewer trailing points.

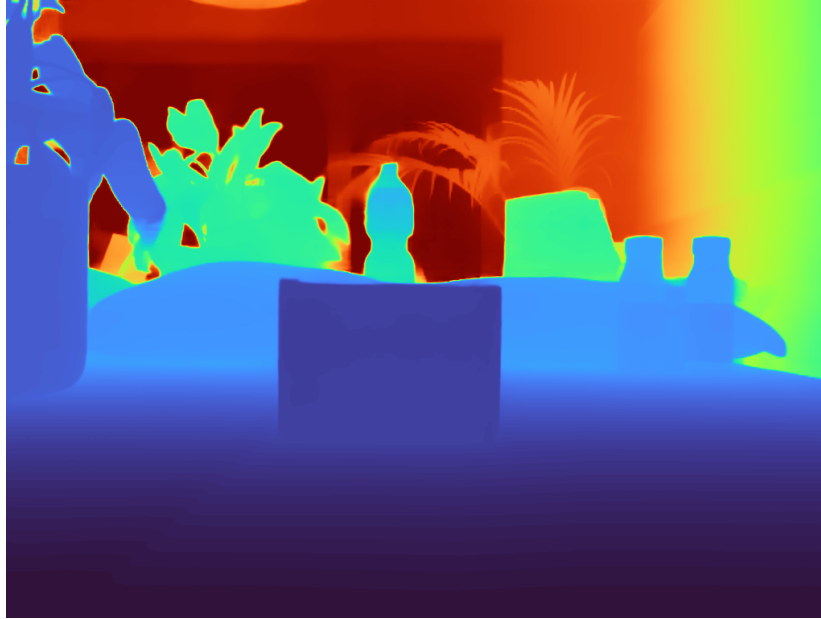


Figure 12: Depth map used to identify depth discontinuities around the foreground box.

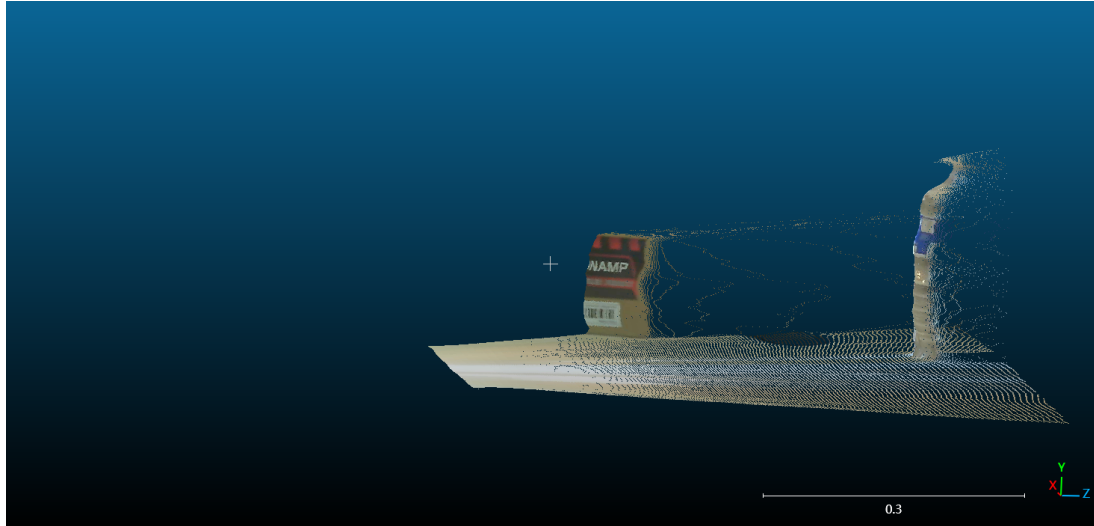
3.4.6 Runtime Performance

The model runs on a GPU. Half-precision inference is enabled to improve speed. Initial model loading takes about 40 seconds. The first image takes about 15 seconds. After that, each later image is processed in about 1.5 seconds. The final setup therefore favours runtime speed, while still keeping point cloud quality at a level that is useful for later stages.

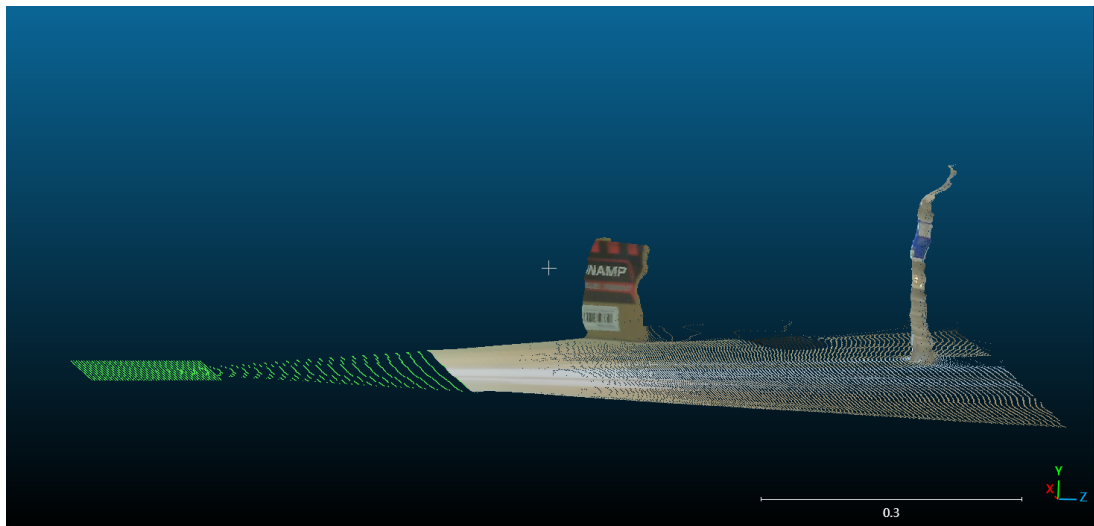
3.4.7 Distance Accuracy Evaluation

The output is evaluated by comparing measurements taken in CloudCompare with real measurements from the physical scene. The main criteria are distance error, contour clarity, and visible shape distortion. In addition, the point cloud is judged by whether it is suitable for later point cloud fusion.

In this evaluation, all tested distances are based on the same target object, which is the small box placed in front of the camera. The measured distance refers to the distance between the camera and the front surface of the box. Table 2 shows the comparison between the real measured distances and the corresponding point cloud measurements for this box at different positions. A visual example of the measurement is shown in Fig. 14.



(a) Before



(b) After

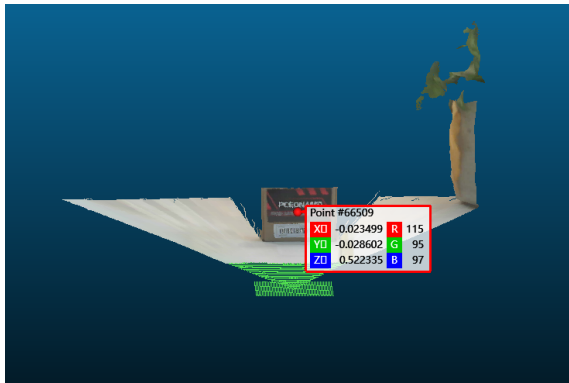
Figure 13: Visual comparison before and after post-processing.

As shown in Table 2, the point cloud is more accurate at shorter distances. From 30 cm to 70 cm, the error stays relatively small. In this range, the absolute error is between 1 cm and 4 cm. This means that the reconstructed point positions are still close to the real scene positions. However, the error becomes larger when the distance increases. At 80 cm, the error rises to 8 cm. At 90 cm, it increases to 10 cm. At 100 cm, it reaches 11 cm. This shows a clear overall trend. The farther the box is from the camera, the less accurate the reconstruction becomes.

Another important point is that the point cloud distance is larger than the real distance at every tested position. This means that the reconstruction tends to overestimate distance in this set of measurements. This overestimation is small at short range, but it becomes more obvious at longer range. For example, the difference is only 1 cm at 60 cm, but it increases to 11 cm at 100 cm. This result suggests that the model output is more

Table 2: Comparison between real measured distances and point cloud measurements for the front surface of the box.

Real distance (cm)	Point cloud distance (cm)	Absolute error (cm)
30	34	4
40	42	2
50	52	2
60	61	1
70	74	4
80	88	8
90	100	10
100	111	11



(a) Point cloud measurement of the box front surface at about 52 cm.



(b) Real scene measurement of the box front surface at 50 cm.

Figure 14: Distance evaluation for the box front surface.

reliable in the near field than in the far field.

The 50 cm case shown in Fig. 14 is one of the measurements already included in Table 2. In this case, the real scene distance is 50 cm, while the point cloud measurement is about 52 cm. The absolute error is therefore 2 cm. This matches the overall trend in the table and gives a direct visual example of the evaluation method.

Based on these results, the final point cloud is limited to a working range of 1 m from the camera. The reason is that the error becomes too large at longer distance. By restricting the effective range, the system can keep better geometric accuracy for later point cloud fusion. A better result is therefore one that gives smaller distance error, clearer object boundaries, and less unwanted deformation within this effective range.

3.5 Point Cloud Merging

3.5.1 System Overview and Mapper Pipeline

The point cloud merging system is developed in a modular manner. Each internal functionality is divided into independent submodules and invoked through concise interfaces. This design improves the extensibility and maintainability of the system, allowing modules to be replaced or extended without affecting the overall structure. At the same time, low coupling between modules is maintained to avoid cascading effects caused by local modifications.

In the overall architecture, the Mapper acts as the core scheduling unit. It does not implement functions but is responsible for organizing and connecting modules to construct a complete processing pipeline.

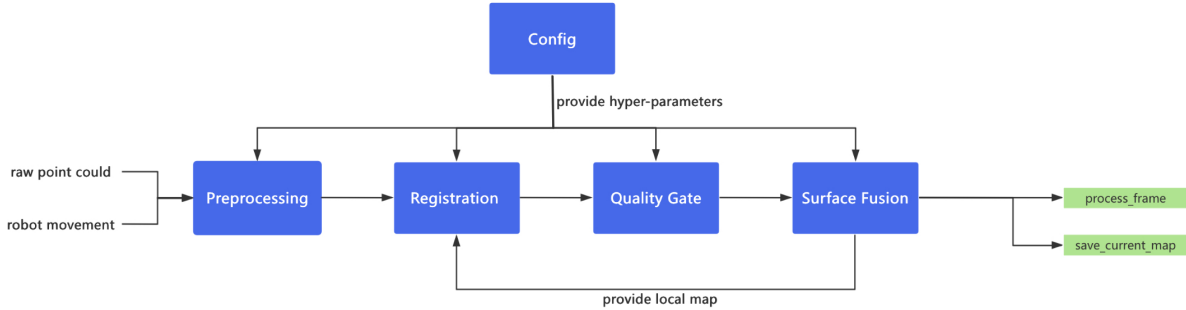


Figure 15: Mapper pipeline schematic.

The Mapper provides two main external interfaces: `process_frame` and `save_current_map`. Among them, `process_frame` is the core interface used to process single-frame input data. This method receives pose estimation and raw point cloud data from upstream modules, and sequentially performs point cloud preprocessing, registration, and surface fusion according to the predefined pipeline, ultimately outputting the optimized pose of the current frame and the updated local map result. The entire process forms an incremental processing mechanism based on single-frame input, enabling the system to construct the global map frame by frame.

The `save_current_map` interface is used to export the currently constructed global map and is typically invoked after data acquisition or processing is completed. Separating real-time processing from result exporting helps ensure that only single-frame data is handled in the main loop, thereby avoiding additional data transfer and computational overhead as the map size grows.

In addition, the system provides a `report` interface for recording and exporting runtime statistical information. This interface collects key metrics after each frame is processed and finally generates a text report file, providing data support for debugging, performance analysis, and result evaluation.

3.5.2 Registration and Supporting Modules

This module refines the pose of the current frame from an externally provided prior transformation through local registration, while incorporating a result gating mechanism to prevent unreliable alignment results from propagating into the map. As shown in Fig. 16, the registration pipeline consists of five stages: prior pose construction, point cloud preprocessing, local map extraction, registration, and result validation.

First, an initial pose is constructed for the current frame. For ordinary frames, this prior is obtained by composing the global pose of the previous frame with the externally estimated inter-frame motion increment. For the first frame, the input pose is used directly. This prior serves as the starting point for the subsequent local registration.

Next, the input point cloud is preprocessed before alignment. This stage includes voxel downsampling, statistical outlier removal, and normal estimation. The purpose of preprocessing is to reduce the computational burden of registration, suppress unstable

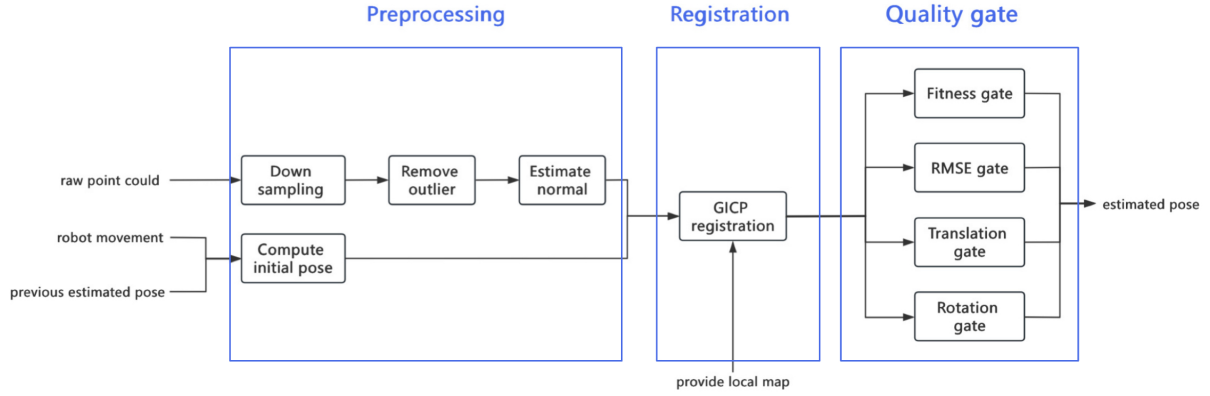


Figure 16: Registration schematic.

observations, and provide reliable local geometric information for the subsequent GICP optimization. The specific parameter settings used in this module are summarized in Table 3.

After preprocessing, a local submap is extracted from the current surfel map around the translational component of the initial pose. Restricting the target point set to a local neighborhood avoids the high computational cost and mismatch risk associated with global-scale registration, while remaining sufficient for local pose refinement.

The registration stage is initialized from the prior pose and solved using GICP, which iteratively estimates the relative rigid transformation between the current frame and the extracted local submap. In this design, the external prior provides the coarse alignment, while GICP performs local geometric refinement around that prior. A coarse registration method based on FPFH and RANSAC with color assistance is also retained in the code-base; however, due to unsatisfactory performance, it is not enabled in the final system, as discussed in Appendix C.

After the GICP optimization, the estimated result is checked by a quality gate. The validation considers both geometric consistency and deviation from the prior pose. If the alignment quality is insufficient, or if the estimated correction is excessively large relative to the initial pose, the registration result is rejected and the system falls back to the prior transformation. This mechanism suppresses the accumulation of local registration failures and improves the robustness of subsequent map fusion.

Finally, the validated pose is used to transform the current frame point cloud into the map coordinate system and pass it to the surfel-based fusion module. In parallel, the Report module records the final pose and gating outcome for later analysis and debugging.

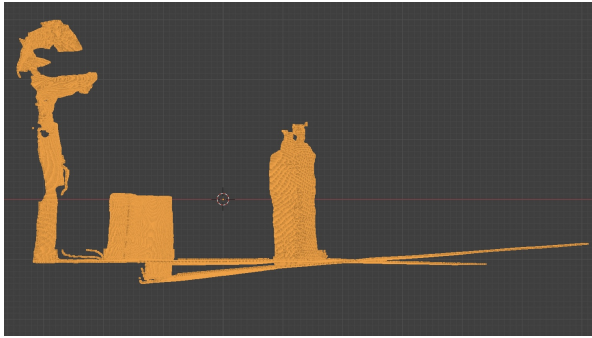
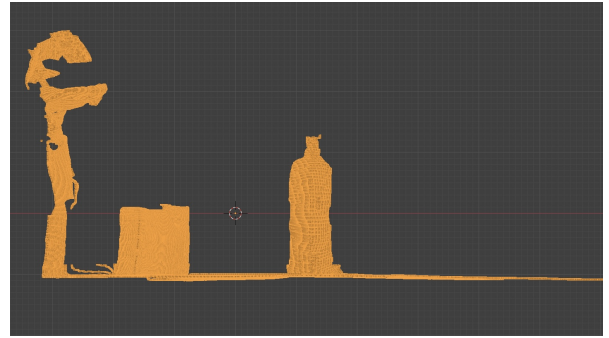
3.5.3 Surface Fusion Module

After registration is completed for the current frame, the surface fusion module incrementally integrates observation points into the global map and continuously updates and maintains it. This module uses surfels as the basic surface representation unit, including geometric attributes, appearance attributes, and observation statistical attributes. The geometric attributes include position and normal, the appearance attributes include color description, and the statistical attributes include accumulated weight, creation frame, last observed frame, and observation count.

Overall, the program adopts a voxel-based spatial organization to manage the global

Table 3: Key hyperparameters of the registration module.

Parameter	Value	Purpose
Voxel size	0.01 m	Preprocessing: downsample points while preserving local geometry
Outlier neighbors	20	Preprocessing: define the local neighborhood for statistical filtering
Outlier std. ratio	5.0	Preprocessing: reject points with large local deviation
Normal estimation radius	0.03 m	Preprocessing: estimate stable local surface normals
Normal max neighbors	30	Preprocessing: limit neighborhood size in normal estimation
Local map radius	3.0 m	Local map extraction: restrict registration to a nearby submap
Max correspondence distance	0.10 m	GICP: suppress distant and likely incorrect matches
Max GICP iterations	100	GICP: limit iterative optimization cost
Minimum fitness	0.40	Quality gate: reject weak geometric alignment
Maximum RMSE	0.08 m	Quality gate: constrain registration residual error
Max translation correction	0.25 m	Quality gate: limit deviation from the prior pose
Max rotation correction	15°	Quality gate: limit excessive rotational correction

**(a)** Without GICP registration.**(b)** With GICP registration.**Figure 17:** Comparison of registration performance.

map. Each voxel maintains a limited number of surfels, balancing local query efficiency and map size control.

During each frame fusion process, the program first constructs an active region based on the current observation area and searches for candidate surfels only within this local region. For each observation point, nearby surfels are searched within a given radius and filtered based on spatial distance, normal consistency, and color similarity. Only when

both geometric and appearance constraints are satisfied are the observation and surfel considered associable. If multiple valid candidates exist, the best match is selected based on combined differences. This strategy aims to reduce computational complexity and the risk of incorrect fusion.

For observation points successfully associated with surfels, the program adopts a consistency-based weighted update strategy to fuse surfel attributes. Let the current observation point be p , the corresponding surfel position be s , the accumulated weight be w , and the current observation weight be w_{obs} . The weight w_{obs} is jointly determined by the distance, normal difference, and color difference between the observation and the surfel, with higher consistency leading to larger weights.

The updated total weight is:

$$w_{\text{new}} = w + w_{\text{obs}} \quad (4)$$

The position is updated using weighted averaging:

$$s_{\text{new}} = \frac{ws + w_{\text{obs}}p}{w_{\text{new}}} \quad (5)$$

Normal and color attributes are updated in the same manner, with normals re-normalized after updating. This mechanism allows stable historical observations to continuously provide prior constraints, while new observations that are more consistent with the surfel exert stronger influence, leading to progressively smoother and more stable surface estimation.

For observation points that cannot be matched to any suitable surfel, the program creates new surfels at the corresponding spatial locations to complement regions not yet sufficiently represented in the map. The initial attributes of new surfels are directly assigned from the current observation and are gradually refined through subsequent multi-frame fusion. In this way, the map can adaptively expand as more observations are incorporated.

To maintain compactness and long-term stability of the map, the program introduces maintenance mechanisms including intra-voxel merging, capacity pruning, and periodic cleaning. For surfels that are spatially close and similar in attributes, merging is performed within voxels to reduce redundancy and improve local consistency. When the number of surfels in a voxel exceeds a predefined limit, surfels with higher observation confidence, greater weights, and more stable states are preferentially retained. Additionally, surfels that have not been updated for a long time or have low weights and insufficient observations are periodically removed to suppress noise and outdated information.

3.6 2D Map Generation and Cost Modeling

To convert the fused 3D point cloud into a representation suitable for path planning, the environment is discretized into regular 2D grid cells [12, 13]. Each cell can only be assigned one of three states: **block**, **costing**, or **unknown**. Here, **block** means that the region is not traversable, **costing** means that the region is traversable but associated with a traversal cost, and **unknown** means that the region has not yet been scanned or has not been reliably observed. The robot is allowed to move only through cells labeled as **costing**. For **costing** cells, the traversal cost ranges from 0 to 255, where a larger value indicates worse road condition and greater traversal difficulty. If the final cost of a cell is greater than or equal to 255, the cell is also treated as **block**.

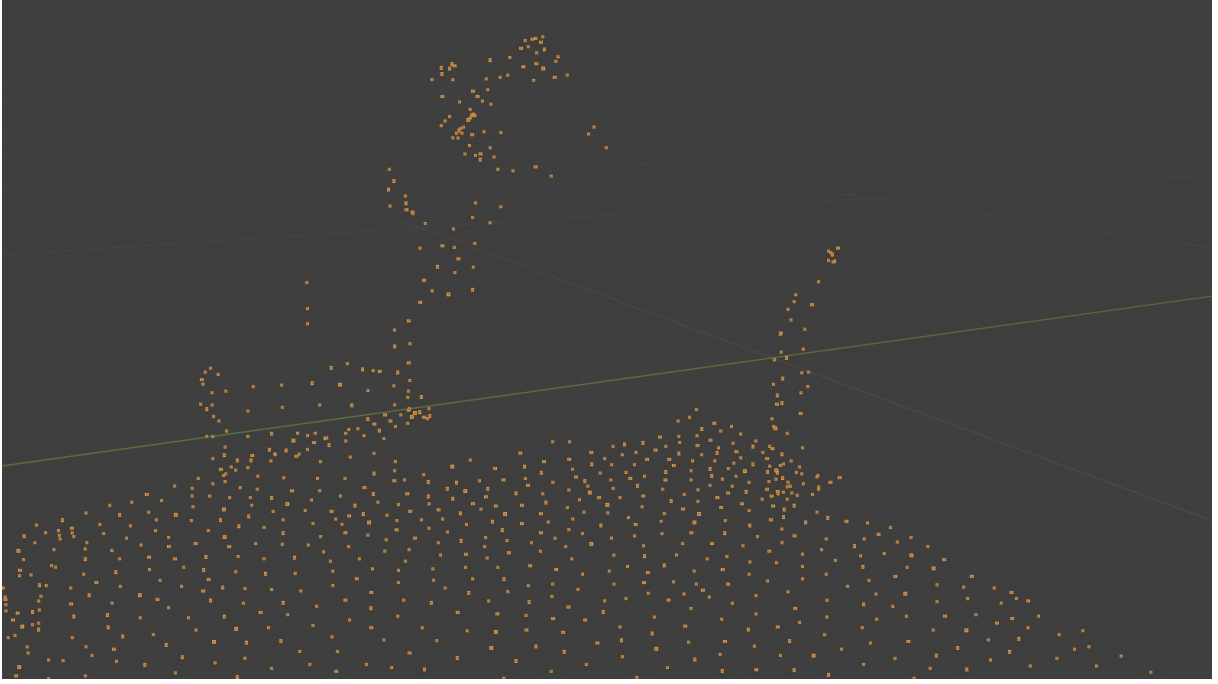


Figure 18: Merged result with surface fusion.

Each grid cell is defined as $5\text{ cm} \times 5\text{ cm}$. This resolution is selected according to the robot size and a safety margin. The robot body is approximately $10 \times 10 \times 11\text{ cm}$, and the diameter of its self-rotation coverage is about

$$2\sqrt{5^2 + 5^2} \approx 14\text{ cm}.$$

To reserve additional redundancy, the robot is approximated as a $15 \times 15 \times 15\text{ cm}$ cube. Under this setting, the $5\text{ cm} \times 5\text{ cm}$ grid provides a convenient resolution for local traversability evaluation. If the neighboring cells covered by the projected robot footprint are all labeled as `costing`, the region can be considered traversable.

3.6.1 Costing computation.

The 2D map is generated by first performing ground-layer selection on the point cloud [14]. After the ground layer is determined for each cell, the system checks whether an obstacle exists within a height threshold H_{th} above the ground layer. If such an obstacle is detected, the cell is directly labeled as `block`. In this work, the height threshold is set to

$$H_{th} = 15\text{ cm},$$

which is slightly larger than the robot height of about 11 cm in order to provide a safety margin.

If no obstacle is found within the threshold, the cell enters the `costing` stage. The traversal cost is computed from both slope and local complexity [13, 14]. The slope is estimated from the local eigenvector [15]. Since the robot can reliably climb slopes up to about 30° , the interval from 0° to 30° is linearly mapped to a cost range of 0 to 255. If the slope exceeds 30° , the corresponding cell is directly labeled as `block`.

In addition, the local complexity is computed from the normalized eigenvalue-based roughness measure [15]. A larger normalized value indicates a more irregular surface.

When the three eigenvalues are equal, the local point set can be regarded as being uniformly distributed in a $5\text{ cm} \times 5\text{ cm} \times 5\text{ cm}$ volume, which corresponds to the maximum complexity. Its normalized value is approximately

$$\sqrt{\frac{1}{3}} \approx 0.6.$$

When the complexity is one tenth of this maximum value, the normalized result becomes

$$\sqrt{\frac{1}{30}} \approx 0.18.$$

Therefore, the complexity threshold is set to 0.18. This threshold is chosen because the robot can stably pass obstacles of about 0.5 cm without significantly affecting pose estimation, so one tenth of the maximum complexity is taken as a practical tolerance level.

The relationship between complexity and actual obstacle severity is not linear, but closer to a quadratic trend. Therefore, the complexity cost is defined as

$$\text{cost}(c) = 255 \left(\frac{c}{0.18} \right)^2,$$

where c is the local complexity value.

3.6.2 Environment types and state conversion.

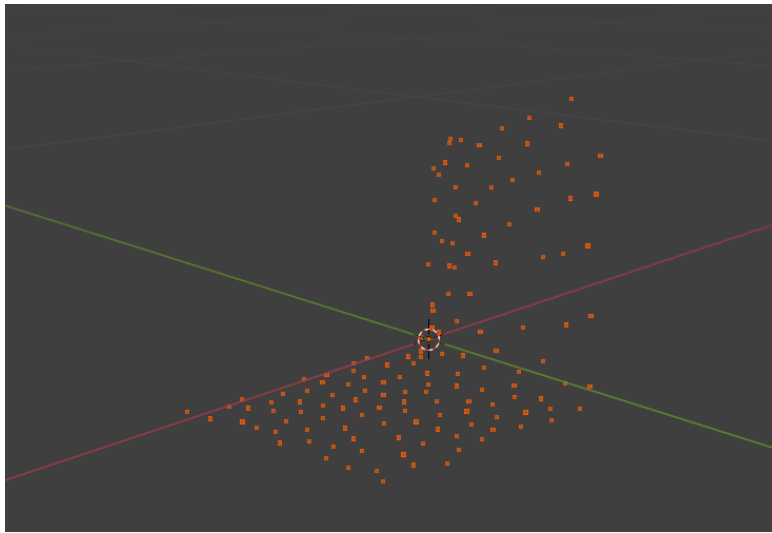
Four typical environment types are considered in this work: walls or steps, suspended obstacles, planar or sloped regions, and pits or cliff-like regions.

Walls, steps, and suspended obstacles are handled in a similar manner. After ground-layer selection, these structures usually appear as elevated obstacles within a limited height range above the detected ground layer. Therefore, once such an obstacle is found within H_{th} , the corresponding cell is directly classified as **block**. Planar regions and slopes are mainly represented through slope and complexity, and are therefore converted into different **costing** values. Small local obstacles are also reflected by increased local complexity, which raises the traversal cost of the corresponding cells [13, 14].

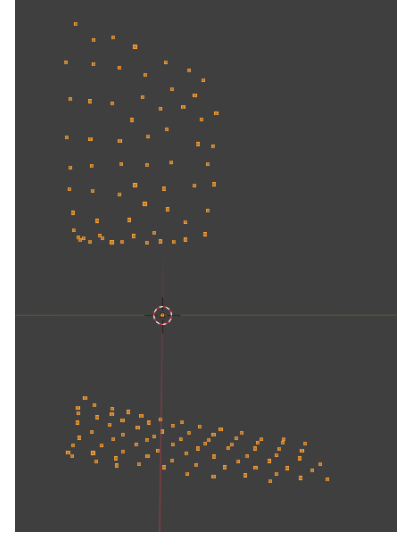
Pits and cliffs exhibit a different behavior. Due to viewpoint limitations, these regions are often difficult to observe completely during early scanning stages, and their edges or walls may not be reconstructed reliably. As a result, such regions are usually labeled as **unknown** first, which prevents the robot from entering potentially dangerous areas. After more observations are collected, these regions may later be reclassified as **costing** or **block**. Typical examples of these environment types are shown in Fig. 19.

3.6.3 Map overwrite and merging.

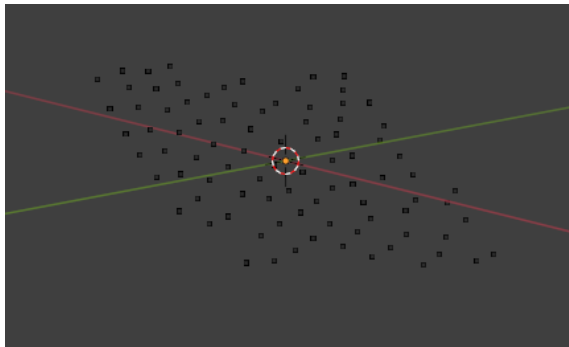
In the global map construction process, each grid cell is associated with its corresponding real-world coordinate. To save computational cost, each frame of map merging updates only the changed point-cloud data, and the 2D map module performs overwrite-based merging on the corresponding cells. This strategy is based on the assumption that, under ideal conditions, repeated point-cloud merging gradually reduces error and produces more accurate geometry. Therefore, newly observed results are used to overwrite previous ones. Through this mechanism, cells originally marked as **unknown** can also be updated into **costing** or **block**, allowing the global 2D map to evolve continuously during the scanning process. A typical overwrite-based update sequence is shown in Fig. 20.



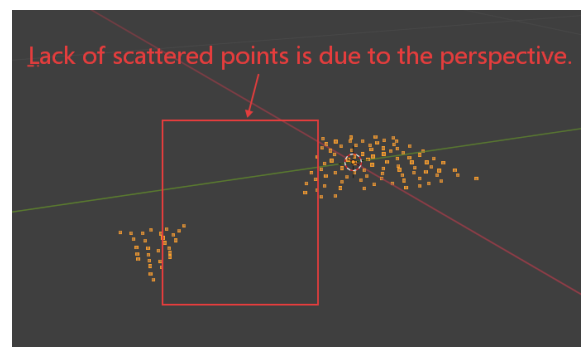
(a) Wall or step obstacle.



(b) Suspended obstacle.

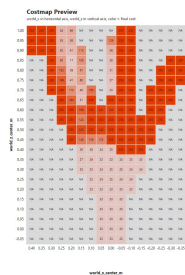


(c) Slope or traversable uneven region.

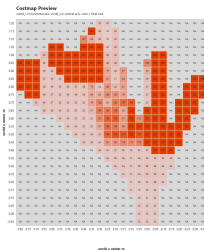


(d) Pit or cliff-like region.

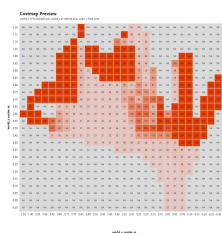
Figure 19: Typical obstacle and terrain types considered in the 2D map generation process.



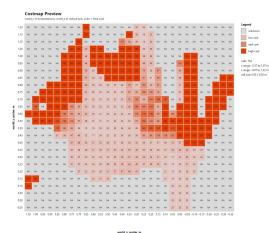
(a) Step 1



(b) Step 2



(c) Step 3



(d) Step 4

Figure 20: Overwrite-based global 2D map update during successive merging steps.

3.7 Path Planning

3.7.1 Path-Planning Overview

This path-planning module enables the robot to start from an unknown environment, gradually expand the known area, and use detected frontier regions as exploration targets until no frontiers remain in the map.

3.7.2 Perception and Map Update

The robot perceives the environment using an FOV model. Within its current facing direction, it uses multiple rays to obtain information about obstacles and traversable areas, and updates the occupancy grid map. At the same time, it performs local updates within a limited range around the robot to improve near-range perception accuracy. The hardware parameters used here are a camera field of view of 45° , a perception range of 8 cells, and a grid size of 5 cm.

To simulate perception delay in real systems, a **MapBuffer** is introduced in the code. During planning, the map from the previous frame is used for decision-making, reflecting the time gap between perception and processing in practical systems.

3.7.3 Path Planning and Decision Making

The input to the path-planning module consists of the cost map in `.npy` format provided by the mapping module, together with the robot's current world pose represented by (x, z, yaw) . The planner first converts the world pose into the corresponding grid position on the current map, and then uses the map cost and free/occupied information to generate the next motion decision.

The output of the planner is not a complete global trajectory, but an executable control sequence of the form

$$\{(c_1, p_1), (c_2, p_2), \dots\}$$

where the first element of each pair is the command type and the second is its parameter. In this system, 0 denotes stop, 1 denotes forward motion, 2 denotes backward motion, 3 denotes left rotation, and 4 denotes right rotation. The parameter is either the forward distance in cm or the rotation angle in degrees.

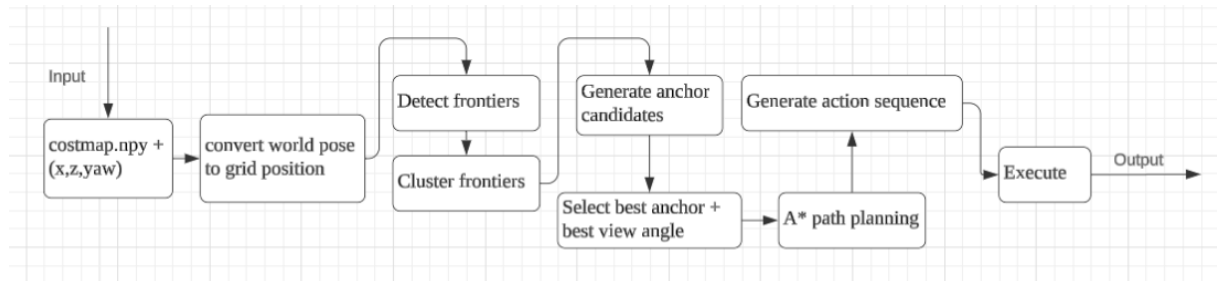


Figure 21: Decision-making pipeline of the robot for each planning step.

The system detects frontier regions near the edge of the current map and connects them into clusters based on 8-neighborhood connectivity. For each frontier region, multiple observation anchors are generated as candidate observation locations. The planner then evaluates these anchors under different viewing angles according to path length, turning cost, map cost, overlap, and anticipated information gain. Once the best anchor is selected, a modified A^* search computes the most feasible path from the current position to that anchor.

3.7.4 Motion and Constraints

The robotic system uses a single-step strategy with a maximum forward movement of 2 cells (10 cm) per step, a maximum rotation of 15° , one-cell obstacle inflation (5 cm), and a minimum observation-overlap ratio of 0.35 in order to support map stitching.

At every step, the system balances whether to scan first or move first, so that the amount of information gained is considered together with the movement cost incurred in the same time interval.

3.7.5 Multi-Robot Coordination

The implemented system consists of a three-robot exploration framework, where all robots contribute to a shared global occupancy grid while performing sensing and decision-making independently.



Figure 22: Multi-robot exploration process over time.

To coordinate the robots, the system uses target exclusion to avoid repeated frontier selection [21], movement constraints and safe-distance rules for collision avoidance [22], priority-based conflict resolution [23], and field-of-view constraints to avoid visual interference [24]. A phased deployment strategy is also adopted so that robots enter the environment gradually rather than all at once [25].

3.7.6 Design Justification and Comparison with Existing Methods

The path-planning module does not use complex optimization algorithms or learning-based methods. Instead, it adopts a cost-aware A^* strategy, which provides a good balance between computational efficiency and planning reliability under the present system setting.

Several commonly used strategies were compared, including A^* -based frontier exploration, random walk, DWA-like methods, VFH-like methods, and D^* Lite. As shown in Fig. 23, the A^* -based nearest-frontier method achieves near-complete coverage while requiring significantly fewer steps than most baseline methods.

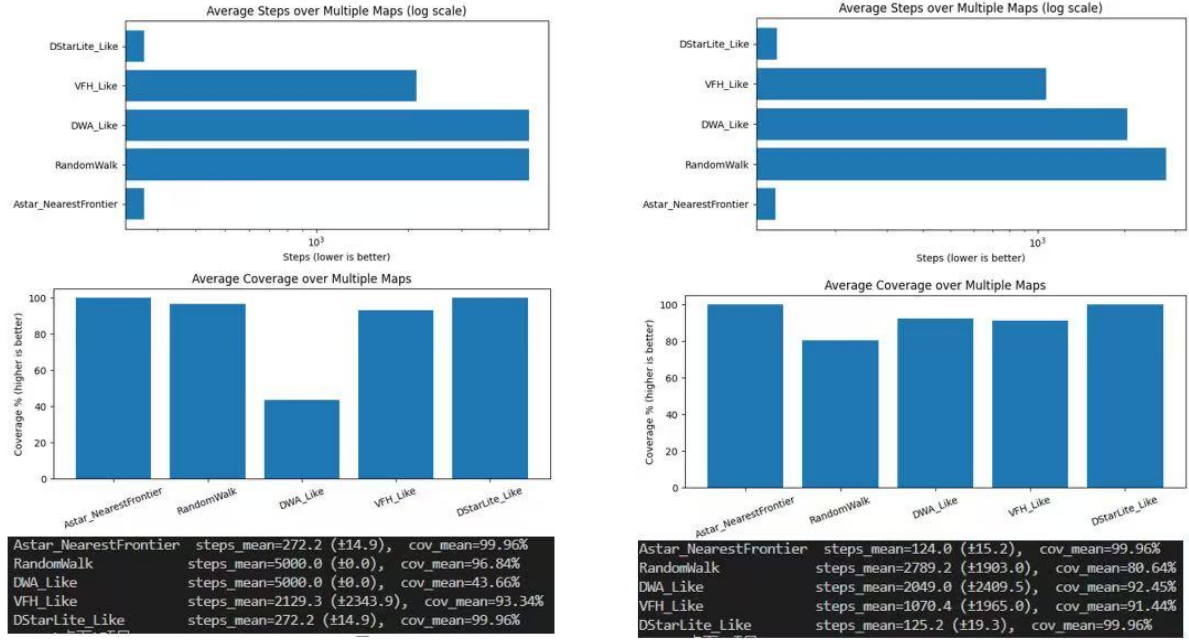


Figure 23: Comparison of exploration performance across different planning methods.

Computational performance was also evaluated in dynamic scenarios. As shown in Fig. 24, A^* requires significantly less total planning time than D^* Lite while maintaining comparable exploration performance.

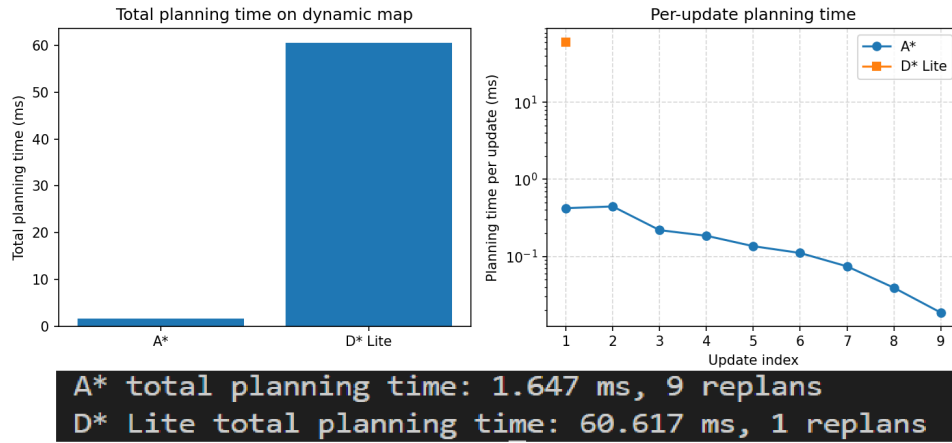


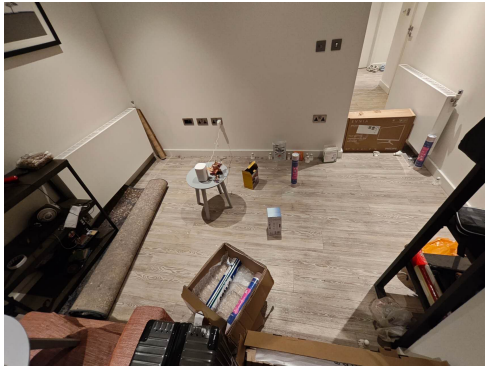
Figure 24: Comparison of total planning time and per-update planning time between A^* and D^* Lite in dynamic environments.

Compared with global optimization or distributed approaches, the current rule-based design is simpler to implement, easier to interpret, and more stable under limited computing resources. At the same time, the explicit inclusion of field-of-view constraints and a perception-delay model makes the planner more representative of real-world deployment conditions.

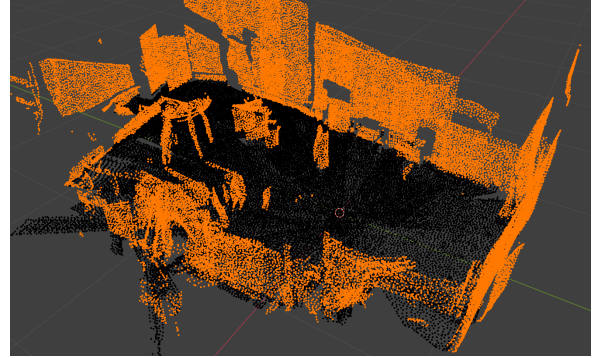
4 Results, analysis, and discussion

4.1 Scanning Result Analysis

The scan result for the 6.7 m² real scene shows that, although the environment is somewhat cluttered and fragmented, the system is still able to represent the approximate spatial relationships among major objects. In overall terms, the reconstructed point cloud preserves the rough arrangement of obstacles and free space and therefore remains usable for subsequent map interpretation. Based on visual inspection of the reconstructed geometry, the practical error is approximately within $\pm 5\text{--}10\text{ cm/m}$.



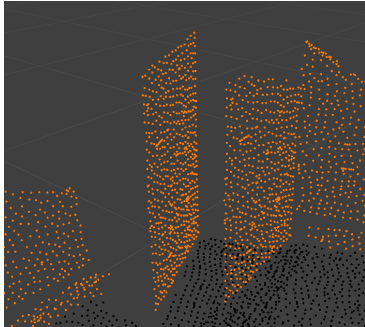
(a) Real indoor scene.



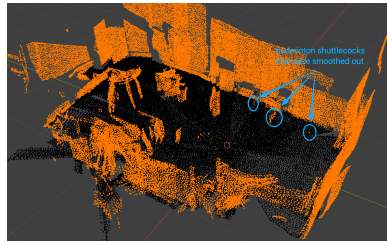
(b) Point cloud of the scan result, where points above the ground layer are highlighted for clarity.

Figure 25: Comparison between the real scene and the reconstructed scan result.

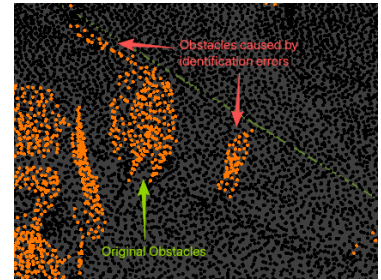
At the same time, several representative errors can be observed in the reconstructed result.



(a) Point-cloud splitting.



(b) A small obstacle smoothed out during fusion.



(c) False obstacles caused by identification errors.

Figure 26: Representative reconstruction errors observed in the scan results.

- **Point-cloud splitting.** Parts of the same physical structure may fail to merge correctly. This issue mainly comes from three causes. First, the point-cloud conversion error for images taken at the same location may already be too large, so the corresponding local structures cannot be aligned reliably. Second, to preserve correct merging in other feature-rich areas, the global merging strategy may accept a compromise that leads to local merging failure. Third, when the robot moves

with an overly large step size, the accumulated pose error becomes too large and the corresponding point clouds can no longer be merged consistently.

- **Small obstacles smoothed out.** If an obstacle is too small, or is too close to surrounding objects, or has visual features too similar to nearby structures, it may be absorbed into neighbouring geometry during the fusion process and therefore disappear from the reconstructed point cloud.
- **False obstacles generated by identification errors.** One cause is motion blur or poor image quality, which makes object boundaries in the estimated depth map unclear. This may create smooth but incorrect depth transitions around object edges, leading to wall-like point artifacts that cannot be removed cleanly by simple sharpening or truncation. Another cause is severe splitting in the merged point cloud, which may create unknown structures that are later interpreted as obstacles.

These error modes can also reinforce one another. When a small obstacle is smoothed out, reliable feature points become harder to detect, which increases the risk of further point-cloud splitting. Conversely, point-cloud splitting may introduce false local features, which can then cause even more splitting in subsequent merging steps. Similarly, falsely generated obstacles may also create artificial features and further destabilise later registration.

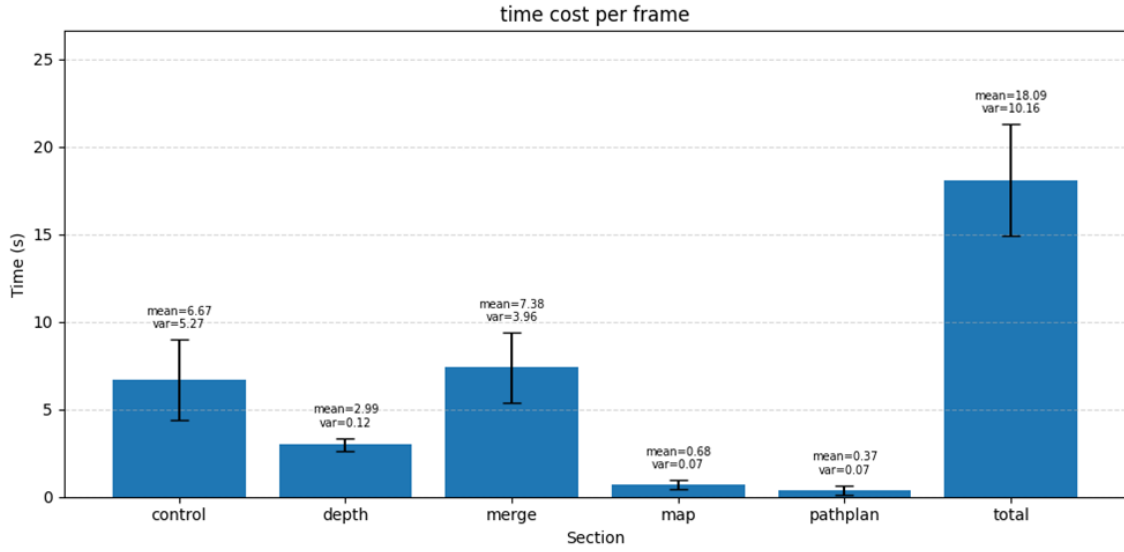
Overall, these problems are strongly related to changes in, or loss of, effective feature points. At a higher level, they mainly originate from two broader causes. One is excessive repeated capture at the same location: if point-cloud conversion already contains noticeable error, repeated observations may repeatedly reinforce the wrong geometry. The other is insufficient capture at the same location: if too few observations are collected, the system may fail to obtain clear and reliable information. Therefore, both too many and too few captures can be harmful. In practice, the most important factor is not simply the number of captures, but the accuracy of the image-to-point-cloud conversion itself, although this accuracy is difficult to guarantee consistently under real scanning conditions.

4.2 Time Consumption and Pipeline Analysis

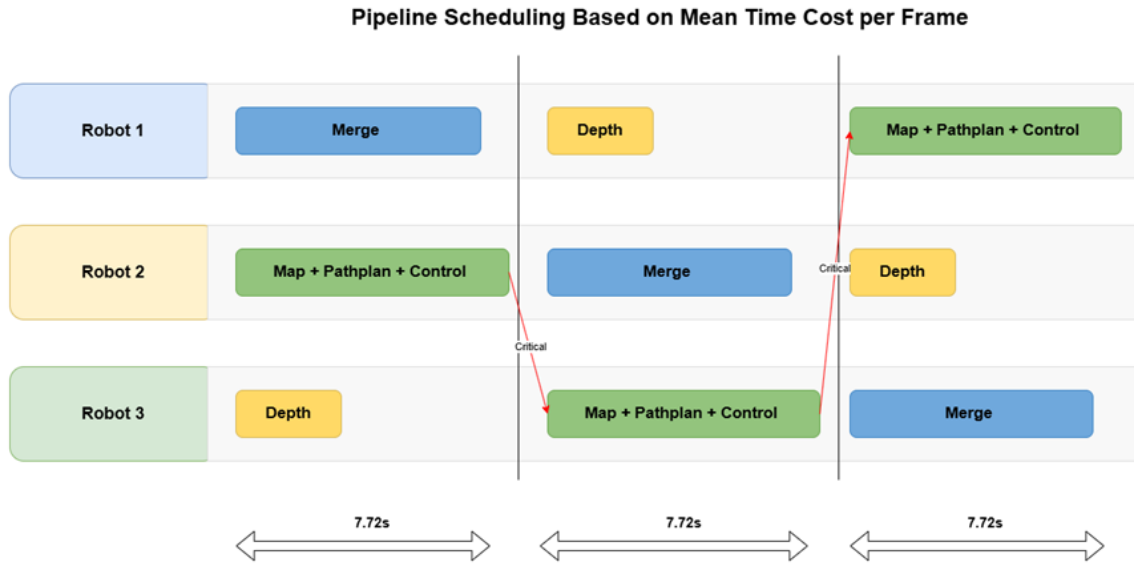
In the runtime environment, the central control host is equipped with an Intel i7-12700H CPU, an NVIDIA RTX 3060 GPU, and 24 GB DDR5 (4800 MHz) RAM. The execution time of each stage in the system, including robot control, image-to-point-cloud conversion, point cloud merging, 2D map projection, and path planning, is measured.

The average processing time per frame is approximately 18.09 seconds. Among all components, the most time-consuming stages are robot control, image-to-point-cloud conversion, and point cloud merging, as illustrated in Fig. 27a.

Although the overall processing time per frame is relatively high, the system benefits from the fact that different modules rely on different hardware resources. This makes it possible to use a pipeline-based parallel computation framework in multi-robot exploration. As shown in Fig. 27b, a new image is generated approximately every 7.72 seconds under the pipeline execution scheme, whereas purely sequential execution would require substantially more time.



(a) Single-frame processing time for the five components of the system.



(b) Pipeline scheduling based on mean time cost per frame.

Figure 27: Time-consumption and pipeline analysis of the system.

4.3 Single-Robot Scene Scanning Experiment

Four experimental environments used in this project are shown together in Fig. 28. Each scenario is constructed by progressively extending the base environment, with Scenario D further introducing an additional corner structure.

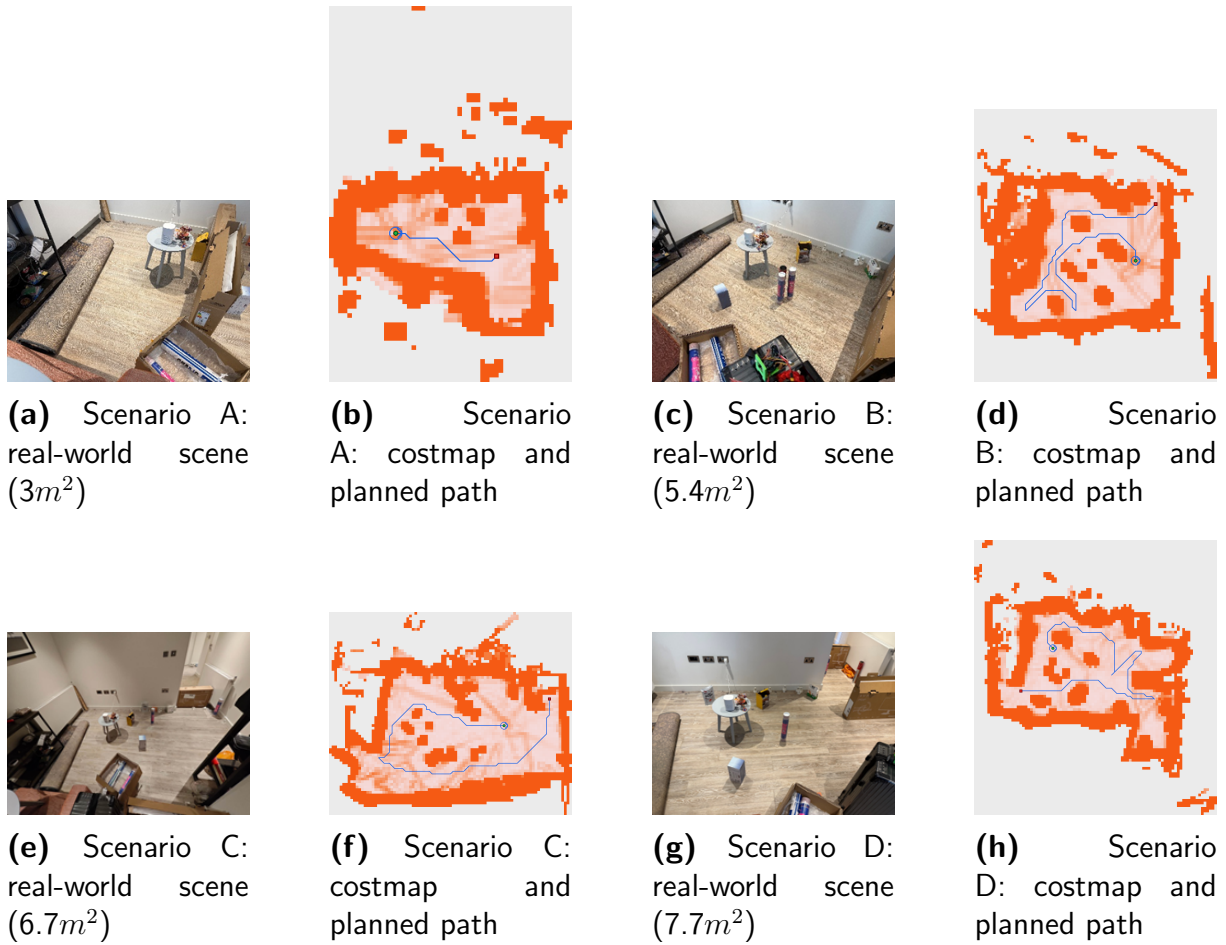


Figure 28: Comparison between the real-world scenes and the corresponding costmap-based path planning results across Scenarios A to D.

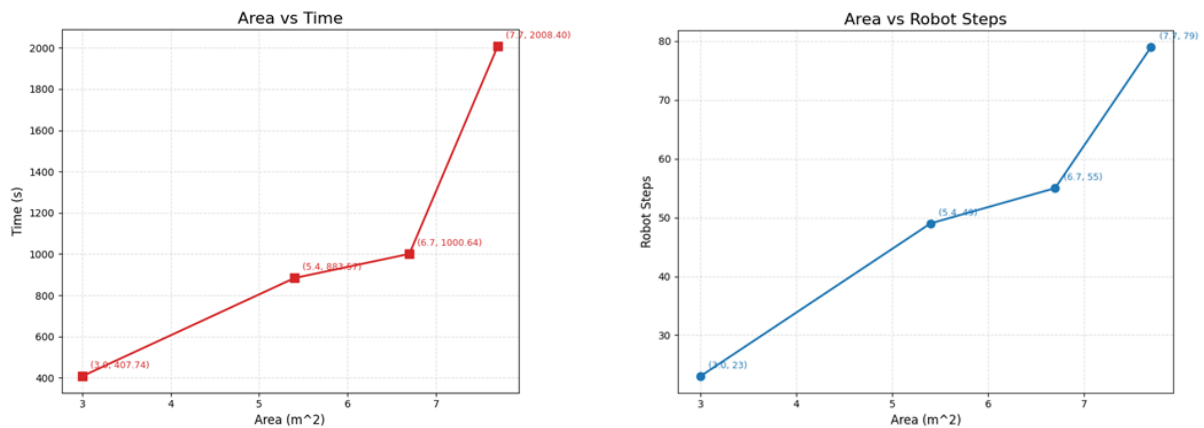
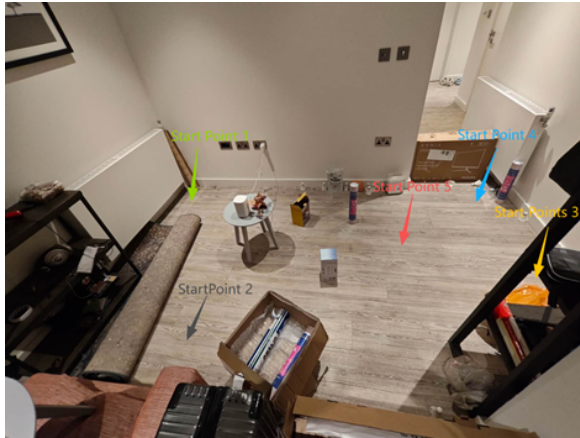


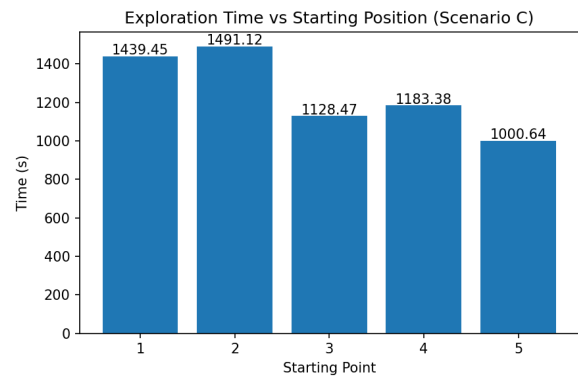
Figure 29: Relationship between environment area and (left) total scanning time, and (right) number of robot steps, where one step denotes one complete program cycle executed to capture a single photograph.

As shown in Fig. 29, the total scanning time is correlated with environment size, but not linearly. Scenario D shows a particularly large increase in total scanning time, mainly

because the additional corner structure makes it harder for the robot to gather information and encourages more locally optimal decisions rather than globally efficient exploration.



(a) Different initial positions of the robot in Scenario C.



(b) Exploration time for different starting positions in Scenario C.

Figure 30: Influence of initial position on exploration performance in Scenario C.

The experiments also show that the initial robot position affects exploration efficiency. A starting point near the center allows the robot to observe a wider area early and acquire richer global information, leading to more effective planning. In contrast, starting positions in more constrained corners tend to increase redundant exploration and may even contribute to repeated scans and later point cloud fragmentation.

4.4 Multiple Vehicles Scanning Simultaneously

As shown in Fig. 31, the relationship between the number of vehicles and scanning speed is not linear. Instead, it exhibits diminishing marginal returns. This effect is especially pronounced in smaller environments, where the available area for independent exploration is limited and coordination overhead becomes dominant.

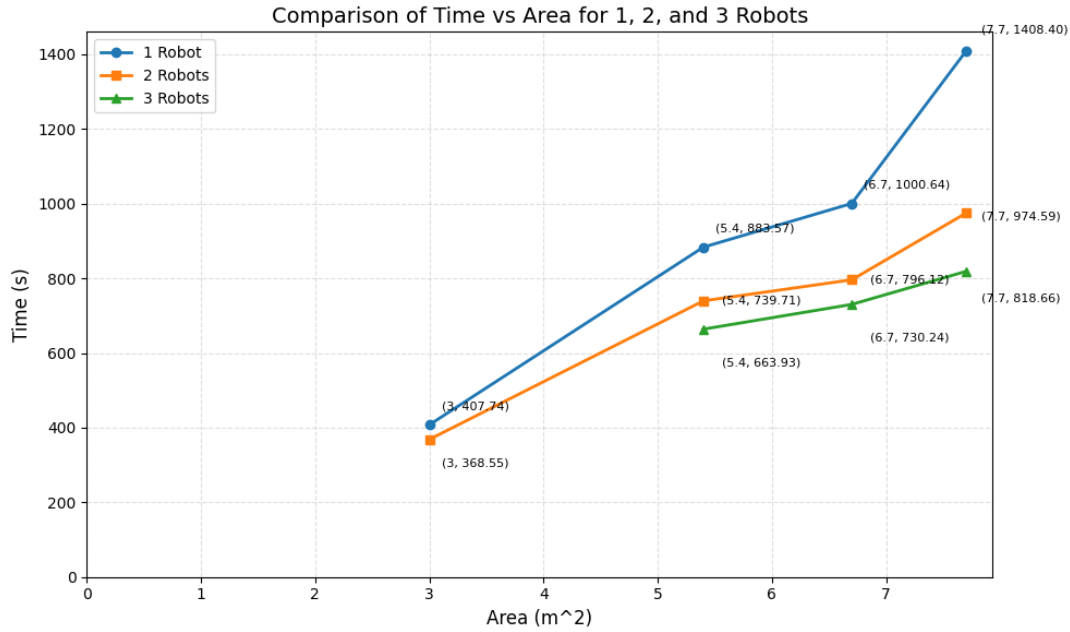


Figure 31: Comparison of time versus area for 1, 2, and 3 robots.

In larger environments, however, the swarm-based multi-robot system has greater effective parallelization capacity. Because multiple robots can scan different subregions simultaneously with less mutual interference, the overall effective coverage per unit time increases, and richer global information is obtained for later path planning and task allocation.

5 Conclusions and Future Work

5.1 Conclusion

This project developed and integrated a low-cost multi-robot scanning system that connects image acquisition, monocular image-to-point-cloud reconstruction, point cloud merging, 2D map generation, and path planning into a complete end-to-end pipeline. The final system shows that useful environment reconstruction and exploration can be achieved without relying on expensive sensing hardware such as LiDAR, and that centralized processing can effectively support multiple small robots while keeping the hardware cost of each unit low.

From the experimental results, several key conclusions can be drawn. First, the image-to-point-cloud module is able to recover geometrically usable local structure within a practical near-field range, although accuracy degrades as distance increases. Second, the point cloud merging and 2D map stages can convert fragmented local observations into a map that is sufficient for traversability reasoning and later planning, but the final quality still depends strongly on point cloud consistency and feature stability. Third, the path-planning experiments show that the frontier-based cost-aware strategy is able to support both single-robot and multi-robot exploration, and that pipeline parallelism improves throughput in the multi-robot setting. At the same time, the results also reveal important limitations, including reconstruction error, local merging failure, false obstacles, single-layer map representation, and coordination overhead in denser multi-robot scenarios.

Overall, the project demonstrates the feasibility of combining monocular reconstruc-

tion, map fusion, and multi-robot path planning in a low-cost framework. However, it also shows that system-level performance is constrained not by a single module alone, but by the interaction between perception accuracy, fusion stability, map representation, and planning decisions. This makes the remaining gaps in the current system both practical engineering challenges and meaningful directions for further research.

5.2 Future Work

The most immediate future work is to improve the geometric reliability of the perception pipeline. The current results show that reconstruction accuracy decreases with distance and that unstable local geometry can later cause point cloud splitting, smoothing of small obstacles, and false obstacle generation. A key research question is therefore how to improve the consistency of monocular point cloud reconstruction across viewpoints while preserving fine local structure. Possible directions include scene-specific fine-tuning of the depth model, stronger post-processing of depth discontinuities, and more explicit confidence modeling during fusion.

The second major direction is to strengthen map representation and fusion scalability. The present system still relies on a single-layer 2D map and a fusion process whose computational cost grows with map size. This limits performance in vertically overlapping environments and in longer scanning sequences. Future work could investigate multi-layer map representations, chunk-based global map storage, and more efficient active-region or spatial-indexing strategies. These directions lead to a broader research question: how can a low-cost monocular system preserve global consistency while remaining computationally efficient in larger and more complex environments?

The third direction concerns planning and coordination. The current cost-aware frontier-based planner works effectively, but the experiments also show repeated paths, local-optimum behavior, and diminishing returns when robot density increases. Future work could therefore study more adaptive target scoring, incremental replanning, and global multi-robot task allocation. An especially important research question is how to balance information gain, overlap requirements, and coordination cost so that additional robots continue to produce meaningful improvements in exploration efficiency rather than mainly increasing system overhead.

More broadly, the results of this project suggest that future progress will depend on tighter coupling between perception, fusion, mapping, and planning rather than isolated optimization of a single module. For this reason, one promising research direction is to design closed-loop strategies in which reconstruction confidence, fusion uncertainty, and planning decisions are updated jointly, so that the system can actively choose motions that improve both map quality and exploration efficiency.

6 Reference

References

- [1] R. Mur-Artal and J. D. Tardós, “Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras,” *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [2] M. Labbé and F. Michaud, “Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

- [3] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2d lidar slam,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, (Stockholm, Sweden), pp. 1271–1278, 2016.
- [4] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, “Digging into self-supervised monocular depth estimation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 3827–3837, 2019.
- [5] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth anything v2,” *arXiv preprint arXiv:2406.09414*, 2024.
- [6] A. Bochkovskii, A. Delaunoy, H. Germain, M. Santos, Y. Zhou, S. R. Richter, and V. Koltun, “Depth pro: Sharp monocular metric depth in less than a second,” *arXiv preprint arXiv:2410.02073*, 2024.
- [7] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge, U.K.: Cambridge University Press, 2 ed., 2004.
- [8] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [9] P. J. Besl and N. D. McKay, “A method for registration of 3-d shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.
- [10] A. Segal, D. Haehnel, and S. Thrun, “Generalized-icp,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2009.
- [11] H. Pfister, M. Zwicker, J. van Baar, and M. Gross, “Surfels: Surface elements as rendering primitives,” in *Proceedings of SIGGRAPH 2000*, 2000.
- [12] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [13] J. Gu, Q. Cao, and Y. Huang, “Rapid traversability assessment in 2.5d grid-based map on rough terrain,” *International Journal of Advanced Robotic Systems*, vol. 5, no. 4, pp. 389–398, 2008.
- [14] X. Meng, Z. Cao, S. Liang, L. Pang, S. Wang, and C. Zhou, “A terrain description method for traversability analysis based on elevation grid map,” *International Journal of Advanced Robotic Systems*, vol. 15, no. 1, pp. 1–12, 2018.
- [15] M. Weinmann, B. Jutzi, and C. Mallet, “Feature relevance assessment for the semantic interpretation of 3d point cloud data,” *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. II-5/W2, pp. 313–318, 2013.
- [16] Espressif Systems, *ESP32 Hardware Reference*. Espressif Systems, 2026. Official hardware reference for ESP32, including datasheet and technical reference links. Accessed: 2026-04-20.

- [17] InvenSense, *MPU-6000 and MPU-6050 Product Specification*. InvenSense, 2013. Revision 3.4, official product specification. Accessed: 2026-04-20.
- [18] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*. STMicroelectronics, 2023. Official datasheet, Rev 5. Accessed: 2026-04-20.
- [19] LCSC, “Lcsc component marketplace.” <https://www.szlcsc.com/>, 2026. Component price source used for the BOM in this work. Accessed: 2026-04-19.
- [20] FreeRTOS, *FreeRTOS Reference Manual*. FreeRTOS, 2025. Official reference manual covering tasks, mutexes, and priority inheritance. Accessed: 2026-04-20.
- [21] M. Zhu, Z. Liu, W. Li, S. Wang, and Q. Zhang, “Improved wavefront frontier detection-utility value task allocation for multi-robot collaborative environmental exploration,” *Automation in Construction*, 2026.
- [22] A. A. Amanatiadis and S. A. Chatzichristofis, “A multi-objective exploration strategy for mobile robots under operational constraints,” in *Proceedings of the IEEE International Conference*, 2013.
- [23] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, and D. Sadigh, “Dynamic multi-robot task allocation under uncertainty and temporal constraints,” *Autonomous Robots*, vol. 46, no. 1, pp. 1–21, 2022.
- [24] M. Catellani and L. Sabattini, “Distributed control of a limited angular field-of-view multi-robot system in communication-denied scenarios: A probabilistic approach,” *IEEE Robotics and Automation Letters*, vol. 8, no. 12, pp. 1–8, 2023.
- [25] G. Best, R. Garg, and J. Keller, “Multi-robot, multi-sensor exploration of multifarious environments with full mission aerial autonomy,” *The International Journal of Robotics Research*, 2024.
- [26] C. M. Sánchez, M. Zella, J. Capitán, and P. J. Marrón, “Efficient traversability mapping for service robots using a point-cloud fast filter (pff),” in *Proceedings of the 19th International Conference on Advanced Robotics (ICAR)*, pp. 584–589, 2019.
- [27] A. E. Carvalho, J. F. Ferreira, and D. Portugal, “3d traversability analysis and path planning based on mechanical effort for ugvs in forest environments,” *Robotics and Autonomous Systems*, vol. 171, p. 104560, 2024.
- [28] Y. Chen, S. Lai, J. Cui, B. Wang, and B. M. Chen, “Gpu-accelerated incremental euclidean distance transform for online motion planning of mobile robots,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6894–6901, 2022.

7 Contribution

7.1 Roles of Team Members in the Project

Zhuyu Dong:

- Design and fabrication of two generations of robot platforms.

- Development of the onboard robot control software and the remote communication interface.
- Refactoring, improvement, and finalization of the point-cloud-to-2D-map module to ensure its successful integration into the overall system.

Junjie Ouyang:

- Image-to-point-cloud reconstruction from captured photographs.

Qingzhou Zhang:

- Point-cloud merging and fusion.

Chi Zou:

- Feasibility-oriented technical exploration of point-cloud-to-2D-map conversion.

Weijie Chen:

- Path planning for the multi-robot system.

7.2 Subsections and Responsible Authors

- Introduction and Literature Review (Section 1) — Weijie Chen, Zhuyu Dong, Junjie Ouyang
- Relative Pose Estimation of the Robot (Section 2.1) — Zhuyu Dong
- Image-to-Point Cloud Reconstruction (Section 2.2) — Junjie Ouyang
- Point Cloud Merging (Section 2.3) — Qingzhou Zhang
- 2D Map Generation and Cost Modeling (Section 2.4) — Zhuyu Dong
- Path Planning (Section 2.5) — Weijie Chen
- Architecture Overview (Section 3.1) — Zhuyu Dong
- Robot Platform (Section 3.2) — Zhuyu Dong
- RTOS-Based Robot Control Architecture (Section 3.3) — Zhuyu Dong
- Image-to-Point Cloud Reconstruction (Section 3.4) — Junjie Ouyang
- Point Cloud Merging (Section 3.5) — Qingzhou Zhang
- 2D Map Generation and Cost Modeling (Section 3.6) — Zhuyu Dong
- Path Planning (Section 3.7) — Weijie Chen
- Results, analysis, and discussion (Section 4) — Weijie Chen, Zhuyu Dong, Junjie Ouyang, Qingzhou Zhang
- Future Work (Section 5.2) — Qingzhou Zhang, Junjie Ouyang, Zhuyu Dong, Weijie Chen

- First-Generation Robot (Section A.1) — Zhuyu Dong
- Second-Generation Platform (Section 3.2.1) — Zhuyu Dong
- Bill of Materials and Cost Estimate (Section 3.2.2) — Zhuyu Dong
- Comparison Between the Two Generations (Section A.2) — Zhuyu Dong
- Image-to-Point-Cloud Reconstruction (Section B.1) — Junjie Ouyang
- Feature Coarse Registration (Section C) — Qingzhou Zhang
- Theoretical, Mathematical and Scientific Background (Section D.1) — Chi Zou
- Methods, Setups and Software Implementation (Section D.2) — Chi Zou
- Quantitative Comparison and Design Analysis (Section D.2.6) — Chi Zou
- Discussion of Robustness and Limitations (Section D.2.7) — Chi Zou

Appendices

A Hardware Iteration of the Robot Platform

1.1 First-Generation Robot

The first-generation robot shared the same overall goal as the final robot platform, but adopted a more function-heavy mechanical structure. Its main controller was an ESP32-WROOM, and the drive system used four ULN2003 drivers with four 28BYJ-48 stepper motors to achieve forward, backward, leftward, and rightward motion. A single front universal wheel was used as the support structure. For image acquisition and active illumination, the robot employed an OV3660 camera and four white LEDs. A dual-servo pan-tilt mechanism was further introduced to provide yaw and pitch adjustment of the camera, so that the robot could scan the environment from multiple viewing directions without changing the body orientation.

To obtain coarse pose feedback, the first-generation robot also included a rotational encoder module for translational distance estimation and an MPU6050 IMU for angular feedback. In principle, this design offered relatively complete functionality. However, practical tests showed several shortcomings. The combination of stepper motors and gear sets occupied significant physical space, which caused the folded robot size to reach about $25 \times 25 \times 15$ cm, and the working-state size to reach about $25 \times 25 \times 35$ cm. Although this was still small compared with many conventional SLAM platforms, it was larger than desired for this project. Moreover, the 1:2 acceleration gear train pushed the stepper motors closer to their load limit, while the omni-wheel structure made slipping more likely and reduced the reliability of pose feedback. On sloped or uneven terrain, the four-wheel omni arrangement could also cause one wheel to lose effective contact with the ground, further reducing motion stability. In addition, the camera pitch mechanism introduced substantial depth-conversion error when images were captured from tilted viewpoints, which reduced the usefulness of such observations for later 3D reconstruction.

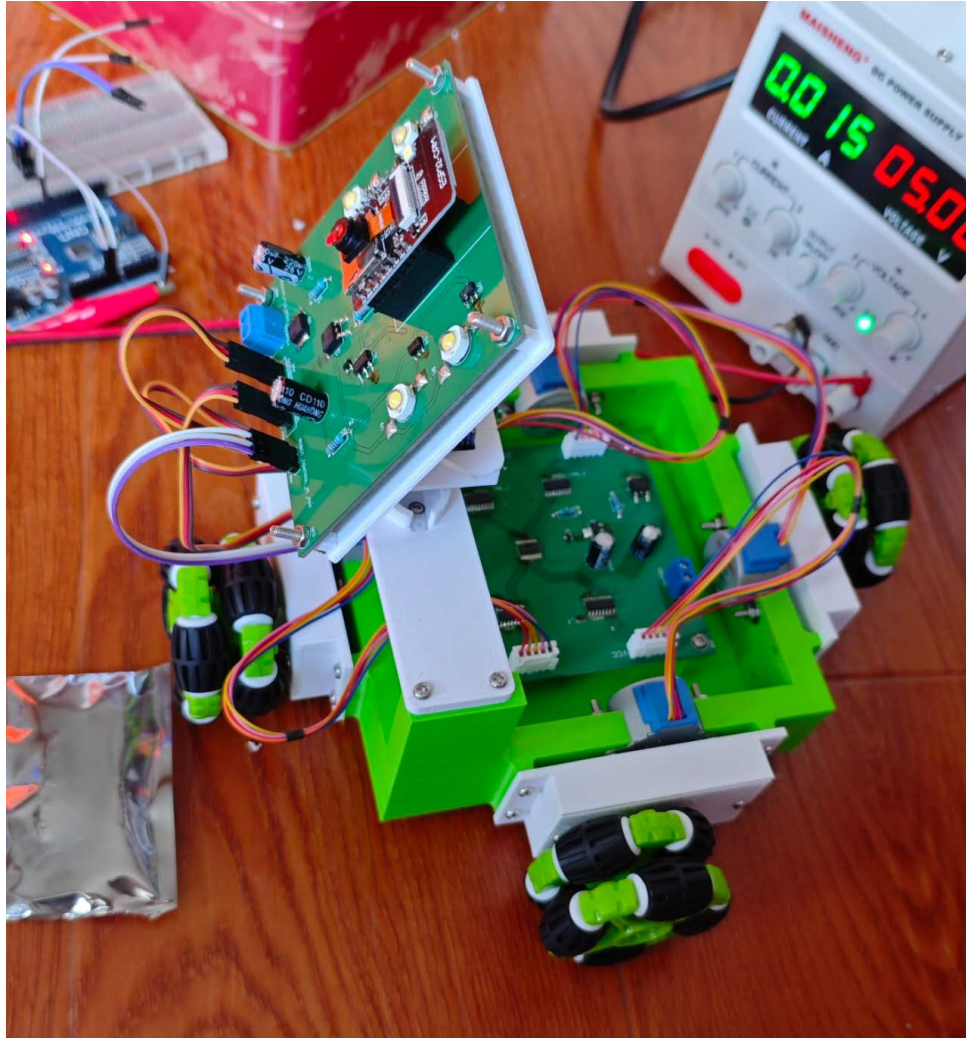
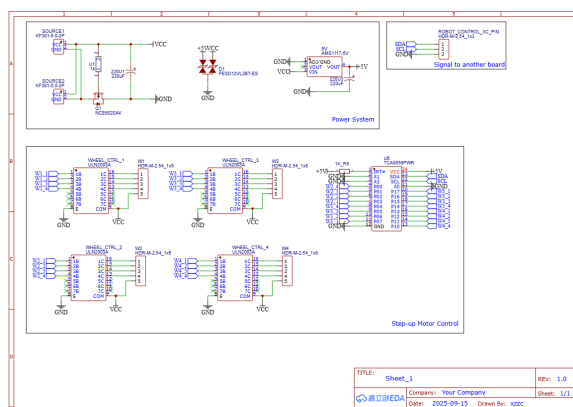
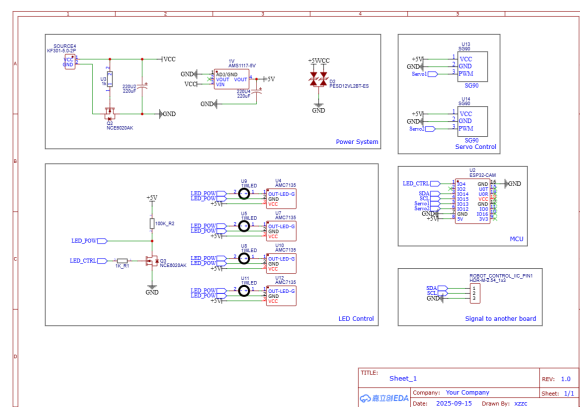


Figure 32: First-generation robot platform.



(a) Body circuit.



(b) Head circuit.

Figure 33: Circuit schematics of the first-generation robot.

1.2 Comparison Between the Two Generations

Overall, the two robot generations reflect two different engineering trade-offs. The first-generation platform emphasised functional completeness, including lateral translation, camera pan-tilt control, and more auxiliary lighting, but this resulted in larger size, higher mechanical complexity, and poorer motion stability. The second-generation robot instead focused on preserving only the core functions required by the project, namely image capture, wireless communication, motion execution, and coarse pose estimation. Through this simplification, the second-generation design achieved better compactness, lower complexity, lower cost, and improved motion accuracy. For these reasons, the second-generation platform was selected as the final hardware platform used in this work.

B Image-to-Point-Cloud Reconstruction Appendix

2.1 Image-to-Point-Cloud Reconstruction

The main implementation file for this submodule is `Final_1280.py`. This file contains the core functions used in this part of the work, including depth estimation, point cloud reconstruction, post-processing, and output generation. It is the main code file used to produce the results presented in this report.

The writing process related to this part of the report is documented in the following repository:

<https://github.com/nuanzhi6666/writing>

This repository contains the writing history of this section, including early drafts, revised versions, and the final written form used in the report. AI tools were used only for grammar checking, correction of language errors, and polishing of wording. The technical ideas, implementation decisions, experimental work, and core written content of this part were developed by the author.

C Feature Coarse Registration

The colour-assisted coarse registration module is designed to provide a more reliable initial alignment for subsequent fine registration. The underlying algorithm is based on coarse registration using local geometric descriptors such as FPFH. Conventional coarse registration primarily establishes candidate correspondences based on local geometric distributions of point clouds. However, when the geometric structure of the input point clouds is distorted, relying solely on geometric features may lead to incorrect matches.

To mitigate this issue, the proposed system introduces a colour consistency constraint on top of geometric feature matching, incorporating local colour statistics as additional discriminative information. Specifically, point cloud colours are first transformed into the Lab colour space, and the a and b channels are extracted as colour descriptors. Compared with the RGB space, the Lab- ab space reduces the influence of illumination variations on matching.

In terms of implementation, the system first performs voxel downsampling with a relatively large voxel size on both the source and target point clouds, followed by normal estimation and FPFH feature computation, in order to construct a sparse set of feature points for coarse registration. Subsequently, for each feature point, the system computes the mean and variance of the Lab- ab colour values within its local neighbourhood, which serve as local colour statistical descriptors.

Candidate correspondences are initially generated based on distances between FPFH geometric features and are further filtered using bidirectional consistency, retaining only mutually selected point pairs. On this basis, the system further evaluates the differences in local colour means and variances between candidate pairs, removing matches with excessive colour discrepancies. The geometric distance and colour difference are then combined through a weighted formulation to re-rank the candidate correspondences.

Finally, a ratio test is applied to eliminate ambiguous matches, and the filtered correspondences are fed into RANSAC to estimate the initial rigid transformation. This design preserves the global search capability of geometric features while enhancing the discriminative power of candidate correspondences through colour information, thereby reducing the risk of mismatches caused by repetitive structures or locally similar geometries.

However, due to unsatisfactory performance observed during experimental evaluation, this module was not enabled in the final system.

D Experimental Point-Cloud-to-2D Map Techniques

This appendix summarizes an experimental point-cloud-to-2D map pipeline developed by Chi Zou. The methods explored in this part focus on converting sparse monocular depth point clouds into semantic and cost-based 2D maps. These techniques were investigated as supporting experiments, but were not incorporated into the final onboard robot workflow used in the main system.

4.1 Theoretical, Mathematical and Scientific Background

4.1.1 Theoretical Basis of 2D Grid-Based Terrain Representation

In mobile robot navigation, although raw 3D point clouds provide a lot of spatial geometric information, performing passability assessments and path planning directly within the full 3D space typically involves processing complex data, leading to high computational costs and long waiting times. Therefore, many studies choose to compress 3D maps into 2D or 2.5D grids, to reduce the complexity of planning and analysis by retaining the topographical information most relevant to movement decisions [14, 26].

4.1.2 Mathematical Definitions of Terrain Geometric Features

In this project, the two-dimensional grid is considered the basic statistical unit for estimating geometric features, rather than simple spatial discretisation units. For each grid cell, the system extracts various geometric features from the near-surface point cloud, including elevation difference, slope and roughness, to assess surface topography and flatness [14, 27].

The following describes how the project processes these geometric features.

- Define the average height near the surface `mean_z` as:

$$\text{mean_z} = \frac{\sum z_i}{N_g} \quad (6)$$

where N_g represents the number of points in a given grid that are classified as belonging to the ground layer.

- The elevation difference between cells `elev_diff` is defined as

$$\text{elev_diff} = z_{\max} - z_{\min} \quad (7)$$

- The roughness is defined as the standard deviation of the elevation of near-surface points

$$\text{roughness} = \sqrt{\max\left(\frac{\sum z_i^2}{N_g} - \mu^2, 0\right)} \quad (8)$$

μ is the average elevation of near-surface points in the current grid.

- Slope is defined from the gradient of the elevation map. In the script, the `mean_z` map is first filled with NaN values, and `np.gradient` is then used to compute the two directional gradients g_x and g_y .

$$\text{slope} = \arctan\left(\sqrt{g_x^2 + g_y^2}\right) \quad (9)$$

NaN is used to mark missing grid cells where no valid ground data is available, and `np.gradient` is used to calculate the discrete gradients in two spatial directions for the interpolated elevation map.

- The script also uses `above_count` to record how many points lie above each grid cell, helping distinguish near-surface obstacles from higher structures.

4.1.3 Theoretical Basis of Semantic Traversability Classification

To make the map easier for robot path planning, this study divides the two-dimensional grid into three categories using semantic labels: UNKNOWN, EXPLORABLE, and BLOCKED [26, 27].

UNKNOWN indicates that there is insufficient observational evidence for the current grid cell; EXPLORABLE means the area has been observed and no explicit obstacles have been detected; and BLOCKED represents the existence of strong geometric evidence proving that the area is not passable.

This three-category classification design is derived from the inherent uncertainty in single-frame depth point clouds, where missing observations, sparsity and noise can lead to certain areas that cannot be classified as either vacant or occupied.

4.1.4 Mathematical Criteria for Hard Blocking

In this study, the BLOCKED area is not defined directly by a single threshold but rather consists of several confirmed hazards. The core theoretical principle is that a grid cell should only be considered non-traversable when it is supported by sufficiently strong geometric evidence [26].

In the code, the final hard block is composed of the following combination:

$$\text{blocked} = \text{obstacle_core} \vee \text{inflated} \vee \text{step_edge} \quad (10)$$

- `obstacle_core` describes distinct vertical obstacles. A grid cell is assigned to this term when it lies in a well-observed area, its `above_count` reaches the minimum obstacle-count threshold, and its maximum near-surface height exceeds the obstacle-height threshold.
- The method uses 8-neighbour support to identify cells whose height differs sharply from several surrounding cells. If this support condition is satisfied, the grid cell is considered a hazardous edge area.

$$|\Delta z| > h_{\text{step}} \quad (11)$$

- **inflated** is the safety expansion of **obstacle_core**, used to represent the robot's physical dimensions and safety margin so that planned paths do not pass too close to obstacle boundaries.

4.1.5 Theoretical Basis of Continuous Traversability Cost

Hard-block reasoning is insufficient for path planning, because traversal difficulty still varies within passable areas. Therefore, this study introduces a continuous traversability cost to represent the relative difficulty of movement within EXPLORABLE regions [27].

For the EXPLORABLE region, the cost is defined as:

$$C = w_s S + w_r R + w_e E + w_p \exp(-d/\lambda) \quad (12)$$

- w_x is the weighting factor in the total cost.
- S is the normalized slope.
- R is the normalized roughness.
- E is the normalized elevation difference within a single cell.
- d is the Euclidean distance from the current grid to the nearest BLOCKED grid.
- λ is the distance decay parameter.
- The proximity term $\exp(-d/\lambda)$ applies a higher penalty near obstacle boundaries and decreases with distance, encouraging safer paths away from hazardous regions [28].

4.1.6 Rationale for Separating Hard Block and Soft Cost

A key design principle of this study is the explicit separation between hard block and soft cost. Hard block is used to determine whether an area is absolutely non-traversable, while soft cost is used to describe the relative traversal difficulty within traversable regions. This design allows the map to represent not only whether a region can be crossed, but also how difficult it is to traverse [27]. Figure 34 gives a visual comparison of these two layers.

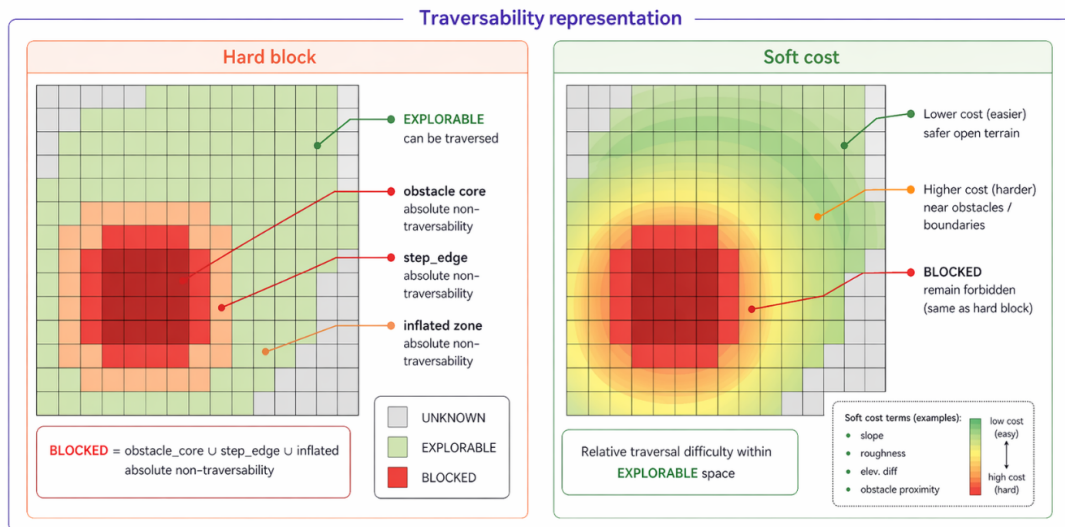


Figure 34: Hard block and soft cost in traversability mapping.

This separation is particularly important for depth-derived point clouds, which are often affected by noise. Since slope is computed from the gradient of a 2D elevation map, it is sensitive to local height errors. If slope is used directly as a hard blocking criterion, traversable areas may be incorrectly marked as blocked. For this reason, the present study restricts BLOCKED regions to confirmed obstacle core, step edge, and their inflated zones, while slope, roughness, and `elev.diff` are retained as soft cost terms in the continuous cost map. In this way, the system reduces over-blocking caused by noise while still preserving the ability to represent local terrain complexity.

4.2 Methods, Setups and Software Implementation

4.2.1 Implementation Pipeline

This experimental implementation converts local point clouds into 2D maps in three stages: the input point cloud is preprocessed and rasterized, a semantic map is generated from near-surface analysis and local geometric features, and a continuous traversability cost map is then constructed.

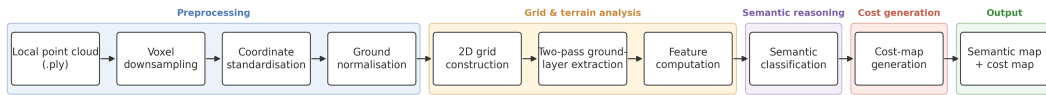


Figure 35: Overview of the proposed pipeline for converting local point clouds into semantic and cost-based 2D maps.

4.2.2 Point-Cloud Preprocessing and Grid Representation

This section takes a `.ply` point cloud file as input and uses the custom script `newmap.py` to convert the local point cloud into a 2D map. The processing workflow first reads the input point cloud, then employs voxel downsampling to reduce point density, limiting the impact of redundant local sampling on subsequent statistical analysis [26, 27]. Then, the `remap_to_z_up()` function is used to normalise the input coordinate system to a Z-axis-up coordinate system, ensuring that height-based feature extraction can proceed smoothly. For point clouds with overall pose deviations, simple ground normalisation and ground plane estimation are further applied to reduce the effects of global skew and height offset.

After normalisation, the point cloud is projected onto a two-dimensional grid. The `res` parameter controls the grid resolution, and a padding operation extends the map boundaries beyond the point cloud’s extent to avoid edge clipping. Each 3D point is assigned to a corresponding 2D cell based on its horizontal position, while vertical information is retained as cell statistics, thereby forming a regular grid representation for subsequent feature computation and semantic mapping.

4.2.3 Ground-Layer Analysis and Semantic Map Generation

After the 2D grid is constructed, not all points within each cell are used directly for classification. The pipeline first extracts near-ground points to reduce the influence of upper-layer structures on ground-related statistics. This study adopts a two-pass strategy for near-ground extraction [27]. In the first pass, the minimum elevation, maximum

elevation, and total point count of each cell are computed. In the second pass, the local minimum elevation is used as a reference, and only points within a predefined ground band are retained as near-ground points, while points above this range are counted separately as `above_count`. Figure 36 illustrates this process.

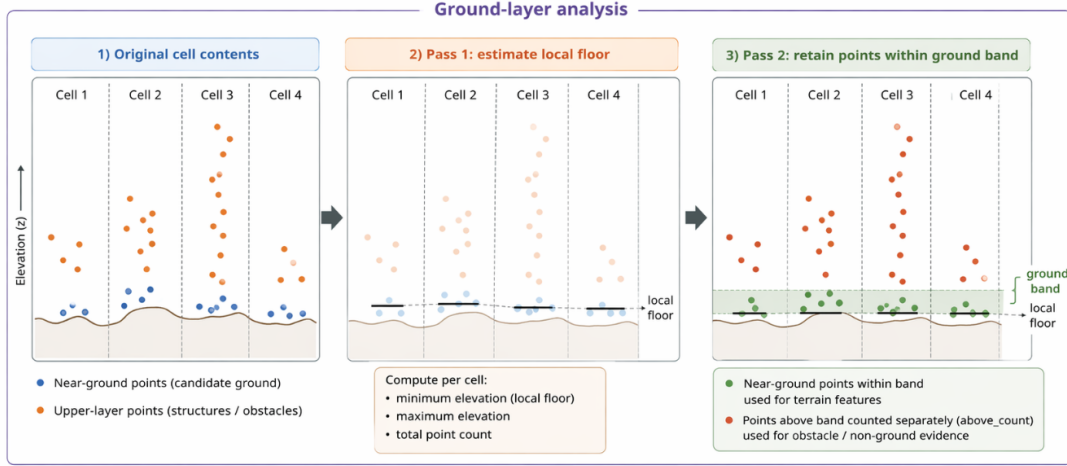


Figure 36: Two-pass ground-layer extraction.

Based on the extracted near-ground layer, the pipeline computes local features, including `mean_z`, `elev_diff`, `roughness`, `slope`, and `above_count`. It then applies an observation quality gate to identify insufficiently observed regions. Cells that satisfy the minimum near-ground point count and local coverage requirements are treated as well observed, while the remaining cells are labelled as UNKNOWN.

For well-observed cells, the final hard block is constructed from `obstacle_core`, `step_edge`, and optional expansion zones. `obstacle_core` represents distinct near-ground vertical obstacles, whereas `step_edge` captures abrupt local height changes. Cells that are well observed but not included in the hard block are labelled EXPLORABLE. A simple cleanup step is finally applied to remove isolated cells and improve the continuity of the semantic map.

4.2.4 Cost-Map Generation

After the semantic map has been generated, the programme proceeds to construct a continuous traversability cost map. The system only calculates continuous costs for EXPLORABLE cells and assigns the maximum cost to both UNKNOWN and BLOCKED cells. The cost map is constructed as a weighted combination of multiple normalised features, including slope, roughness, `elev_diff`, and an obstacle proximity term derived from the distance transform [28]. In this way, the cost map not only preserves the discrete semantic boundaries but also represents the relative difficulty of traversal within passable regions.

4.2.5 Representative Map Outputs and Implementation Evaluation

Figure 37 shows the semantic map for scene `frame_5_D` and its corresponding traversability cost map. After preprocessing, the scene is represented on a 37×30 grid with a cell resolution of 0.05 m. The semantic map shows that UNKNOWN mainly occupies peripheral regions with limited observation, while EXPLORABLE appears near the sensor

origin and BLOCKED is concentrated around obstacle boundaries and step-like edges. This distribution is consistent with the proposed observation-gated and step-edge-based classification design.

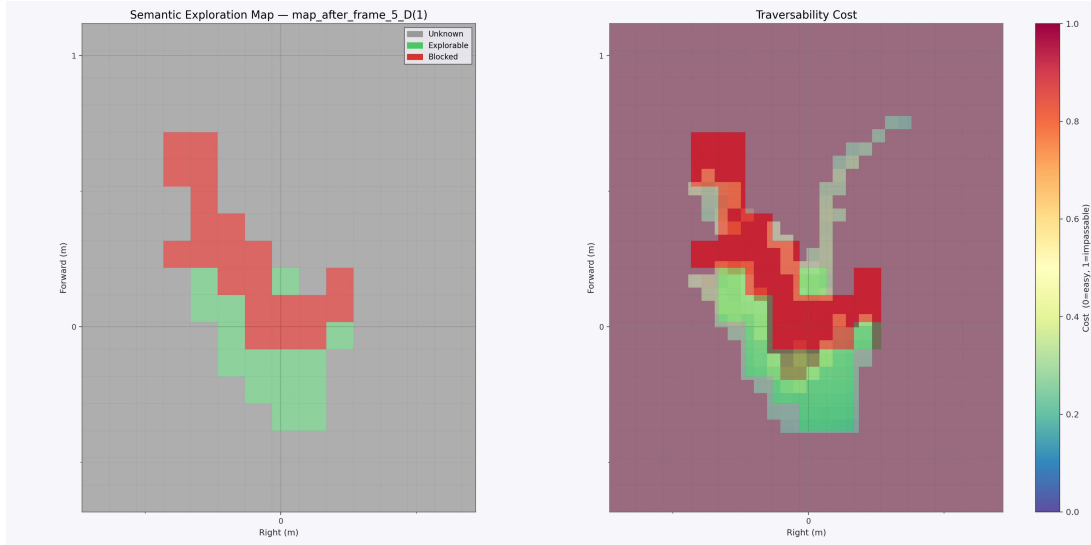


Figure 37: Semantic exploration map and traversability cost map for scene `frame_5_D`.

The corresponding cost map exhibits a smooth cost gradient within the EXPLORABLE region. Costs are lower in cells farther from BLOCKED areas and higher near obstacle boundaries. This behaviour is consistent with the proximity term derived from the distance transform.

Figure 38 presents three additional semantic maps that illustrate the range of conditions in the test set.

- Scene `frame_6_S` contains a larger proportion of valid near-ground observations and therefore a wider EXPLORABLE region.
- Scene `map (1)` shows a corridor-like layout, where BLOCKED is mainly concentrated along the side boundaries, leaving a continuous central passage.
- By contrast, scene `frame_1_D` represents a sparse and noisy case, in which most cells remain UNKNOWN.

These examples indicate that the pipeline preserves traversable structure when observations are sufficient, while remaining conservative when evidence is limited.

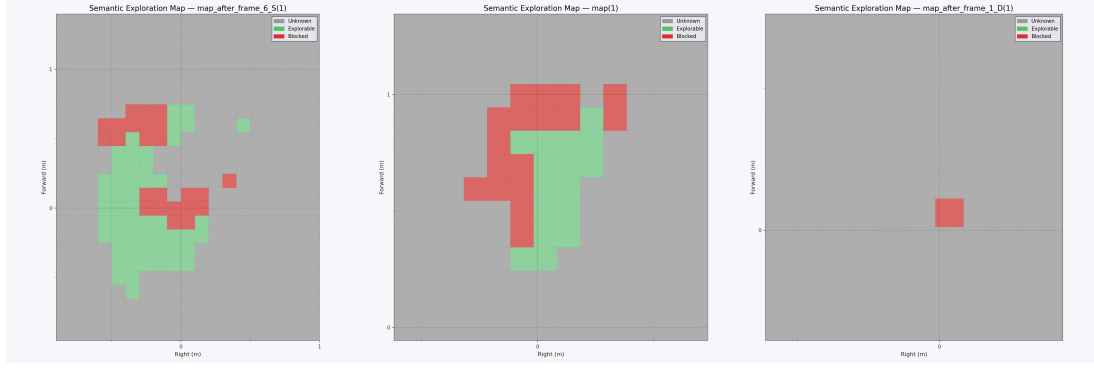


Figure 38: Additional semantic-map examples from representative scenes in the test set.

4.2.6 Quantitative Comparison and Design Analysis

Scene	Grid (H × W)	UNKNOWN %	EXPLORABLE %	BLOCKED %
map(1)	29 × 28	81.90	13.92	4.19
map(3)	36 × 28	81.65	16.07	2.28
map_after_frame_1_D	24 × 22	95.45	4.36	0.19
map_after_frame_1_S	19 × 20	95.26	2.63	2.11
map_after_frame_2_D	30 × 26	92.82	5.51	1.67
map_after_frame_2_S	30 × 26	88.85	9.10	2.05
map_after_frame_3_D	32 × 26	90.26	6.01	3.73
map_after_frame_3_S	30 × 26	88.85	9.10	2.05
map_after_frame_4_D	34 × 27	88.24	8.06	3.70
map_after_frame_4_S	36 × 30	85.09	13.06	1.85
map_after_frame_5_D	37 × 30	84.14	11.44	4.41
map_after_frame_5_S	44 × 39	82.93	14.86	2.21
map_after_frame_6_S	48 × 39	79.59	17.68	2.72
Mean (n=13)	—	87.31	10.14	2.55
Median (n=13)	—	88.24	9.10	2.21

Table 4: Class distribution across the 13-scene test set under the default parameter setting.

Table 4 summarises the class distribution across the 13-scene test set under the default parameter setting. Across the dataset, UNKNOWN remains the dominant class, while EXPLORABLE and BLOCKED occupy relatively small proportions. This is consistent with the single-frame nature of monocular depth input, where only part of the environment is observed in each frame and the observation gate therefore retains a large proportion of cells as UNKNOWN. The limited size of the BLOCKED set indicates that the current pipeline does not over-label non-traversable areas when evidence is sparse.

traversable_step_height (m)	UNKNOWN %	EXPLORABLE %	BLOCKED %	Δ BLOCKED vs default
0.04	84.05	9.73	6.22	+1.81 pp
0.06	84.14	10.36	5.50	+1.09 pp
0.08 (default)	84.14	11.44	4.41	—
0.10	84.14	12.07	3.78	−0.63 pp
0.12	84.05	12.34	3.60	−0.81 pp

Table 5: Sensitivity of class distribution to traversable_step_height in scene frame_5_D.

min_step_support	UNKNOWN %	EXPLORABLE %	BLOCKED %	Δ BLOCKED vs default
2	84.05	10.81	5.14	+0.73 pp
3 (default)	84.14	11.44	4.41	—
4	84.05	12.16	3.78	−0.63 pp
5	83.96	12.97	3.06	−1.35 pp

Table 6: Sensitivity of class distribution to min_step_support in scene frame_5_D.

Table 5 and Table 6 show that increasing traversable_step_height or min_step_support leads to a monotonic decrease in the proportion of BLOCKED cells, while UNKNOWN remains largely unchanged. This suggests that these parameters mainly control the strictness of step-edge detection rather than the behaviour of the observation gate. By contrast, changing min_obs_neighbours has little effect on the final class distribution.

Configuration	UNKNOWN %	EXPLORABLE %	BLOCKED %
Slope hard-block, $> 15^\circ$	83.96	5.23	10.81
Slope hard-block, $> 20^\circ$	83.96	5.77	10.27
Slope hard-block, $> 30^\circ$	84.05	8.11	7.84
Slope hard-block, $> 40^\circ$	84.05	9.46	6.49
Slope hard-block, $> 50^\circ$	84.05	10.90	5.05
Slope as soft cost (current)	84.14	11.44	4.41

Table 7: Effect of reintroducing slope as a hard-blocking criterion in scene frame_5_D.

Table 7 further discusses the treatment of slope in the current design. When slope is reintroduced as a hard-blocking criterion, the proportion of BLOCKED cells increases substantially and the EXPLORABLE region is correspondingly reduced. This suggests that slope is highly sensitive to local discretisation and height noise in the present depth-derived point clouds and would therefore introduce excessive false positives if used as a hard block.

Overall, these results support the current design choice of reserving hard blocking for stronger geometric evidence, while retaining slope in the continuous cost map as a soft penalty.

4.2.7 Discussion of Robustness and Limitations

These tests show that the current method is capable of generating 2D maps from single-frame depth-derived point clouds. The UNKNOWN regions are primarily influenced

by the quality of the observations, and the BLOCKED regions mainly appear near the boundaries of obstacles and the edges of steps. These results are consistent with the expected design. However, as this method was evaluated using only 13 sets of monocular depth point clouds and did not employ external ground truth, the analysis focuses on internal consistency rather than absolute accuracy. In sparse or noisy scenes, large areas of UNKNOWN may remain, which may require recalibration of the parameters.

E Repositories Related to This Work

Zhuyu Dong:

- **Project code repositories:**

<https://github.com/SlamRobotV3/RobotESP32.git>

<https://github.com/SlamRobotV3/ControlUtils.git>

https://github.com/SlamRobotV3/2D_map.git

<https://github.com/SlamRobotV3/SUM.git>

- **Report writing repository:**

<https://github.com/xzzc666/project-2-final-report.git>

Junjie Ouyang:

- **Project code repository:**

https://github.com/SlamRobotV3/Image_to_3D

- **Report writing repository:**

<https://github.com/nuanzhi6666/writing>

Qingzhou Zhang:

- **Project code repository:**

<https://github.com/SlamRobotV3/Merging.git>

Chi Zou:

- **Project code repository:**

<https://github.com/SlamRobotV3/2D-map-creation.git>

Weijie Chen:

- **Project code repository:**

<https://github.com/SlamRobotV3/pathplan.git>

- **Report writing repository:**

<https://github.com/ntjj6/final-report>

F AI Assistance Statement

OpenAI's GPT-5.4 and GPT-5.3 Codex were used as auxiliary tools during the development of this project and the preparation of this report. Their main uses included code assistance during implementation, support for idea expansion and organization during project planning, and assistance with translation, grammatical polishing, and length reduction during report writing. However, the main technical ideas, system design decisions, implementation work, and core content of the project were developed by the author. Drafts and iteration details related to the writing process can be examined through the commit history of the repositories listed in the previous section.